

Group number 7

조원

김주하 (20231561)

유지원 (20232095)

이재민 (20212861)

최은지 (20231914)

Computer Architecture

Project – Architectural Simulation

숭실대학교 전자정보공학부 · SimpleScalar 기반 프로세서 / 캐시 구성 성능 분석

목차

목차	2
1. 프로젝트 개요	5
(1) 프로젝트 수행 이유	5
(2) 수행을 통해 얻을 수 있는 정보	5
성능 측정	5
application 별 최적화 정보	5
(3) simulation의 중요성	5
설계 예측	5
설계 검증	5
비용 절감	5
2. 주요 configuration 분석	7
(1) Processor 파이프라인 파라미터	7
(2) bpred: branch predictor type	7
(3) cache	8
(4) res	8
3. Processor 변화에 따른 성능 분석	10
(1) 연산 장치 수에 따른 성능 분석	10
1. 연산장치 수에 영향을 미치는 configuration	10
2. 성능 비교를 위해 선택한 Simulation Statistics	10
3. 실행 결과 그래프	10
4. 결과 분석	11
5. trouble shooting	12
(2) Super-scalar에서 scalar 수에 따른 성능 분석	13
1. scalar의 수에 영향을 미치는 configuration	13
2. 성능 비교를 위해 선택한 Simulation Statistics	13
3. 실행 결과 그래프	13
4. 결과 분석	14
(3) Branch predictor 종류에 따른 성능 분석	16
1. bpred 방식에 따른 결과	16
2. 성능 비교를 위해 선택한 Simulation Statistics	16
3. 실행 결과 그래프	16
4. 결과 분석	17
(1) GCC / ANAGRAM / GO — 3-bit이 default(bimod)보다 우수	17
(2) COMPRESS95 — 1-bit이 default(bimod)보다 우수	18
4. Cache 변화에 따른 성능 분석	19
(1) Cache block size 크기에 따른 성능 분석	19
1. 실행 결과 그래프	19
측정 데이터	20

2. 결과 분석	21
3. Trouble shooting	21
(2) Cache associativity에 따른 성능 분석	23
1. 실행 결과 그래프	23
측정 데이터	24
2. 결과 분석	25
(3) Cache replacement policy에 따른 성능 분석	26
1. cache replacement policy	26
이론에 따른 simulation 예측	26
2. 성능 비교 시 주요 configuration 선택	26
(1) dl1, dl2 miss rate	26
(2) CPI	26
성능 비교 방법 — AMAT (Average Memory Access Time)	27
3. application 별 분석	28
I. GCC	28
(L-L) 정책의 단점	29
II. ANAGRAM	30
(L-R) 정책의 단점	31
III. COMPRESS95	32
COMPRESS95의 워크로드 특성	33
(L-L, F-F) 정책의 단점	33
IV. GO	34
(L-R) 정책 단점	35
(L-L) 정책 단점	35
결론	35
5. 최적의 cache configuration 탐색	36
1. simulation 방법	36
simulation 방법 선택 이유	36
2. simulation 결과 (캐시 크기: dl1=128, dl2=1024)	36
I. GCC	36
실행 결과 분석	38
II. ANAGRAM	39
실행 결과 분석	40
III. COMPRESS95	41
(1) trouble shooting	41
(2) 실행 결과 그래프	42
(3) 실행 결과 분석	43
IV. GO	44
(1) trouble shooting	44
(2) 실행 결과 그래프 (수정 후)	45

(3) 실행 결과 분석	46
--------------------	----

1. 프로젝트 개요

(1) 프로젝트 수행 이유

강의를 통해 학습한 이론적 개념을 실제로 simulation해 결과를 분석하고 해석하는 과정을 통해 컴퓨터 구조 설계의 원리와 효과적인 하드웨어 설계 방법에 대한 이해를 심화한다. 이를 통해 이론과 실제 하드웨어 설계를 연결 짓는 능력을 키울 수 있으며, 문제 해결 능력을 발전시킬 수 있다. 또한 팀원들과 협업을 통해 문제를 함께 해결하고, 다양한 관점을 반영해 최적의 설계 방향을 도출하는 경험을 쌓을 수 있다. 이 과정에서 의사소통 능력과 프로젝트의 진행 과정을 효율적으로 분배, 관리하는 능력을 기를 수 있다.

(2) 수행을 통해 얻을 수 있는 정보

성능 측정

다양한 설계 옵션에 대한 simulation을 통해 기본적인 성능 지표(CPI, cycles 등)를 확인할 수 있다. 이를 통해 설계 옵션 간의 성능 차이를 확인할 수 있으며, 이를 분석해 최적화에 대한 정보를 얻을 수 있다. 이는 분석 과정을 거쳐 실제 설계 개선의 기초 자료로 활용될 수 있다.

application 별 최적화 정보

특정 application에 적합한 최적화 정보를 얻어 실제 성능을 극대화할 수 있는 설계 방향을 설정할 수 있다. 이를 통해 특정 워크로드(주어진 시간 안에 컴퓨터 시스템이 처리해야 하는 작업의 양이나 유형)에 특화된 하드웨어 설계를 구현하는데 기여할 수 있다.

(3) simulation 의 중요성

설계 예측

다양한 워크로드에서 설계된 시스템의 성능을 예측하고 실제 구현에서의 작동 방식을 미리 파악할 수 있다. 이를 통해 설계의 방향성을 설정할 수 있다.

설계 검증

실제 하드웨어를 제작하기 전에 설계된 컴퓨터 구조를 가상 환경에서 테스트하여 실제 설계의 정확성 및 안정성을 확인할 수 있다. 또한 오류나 병목 현상 등을 조기에 발견해 안정적인 하드웨어를 개발할 수 있다.

비용 절감

하드웨어를 물리적으로 제작하지 않고 다양한 설계 옵션(연산 장치의 수, 캐시 크기 등)을 비교해 최적의 구조를 제작할 수 있다. 이를 통해 불필요한 비용의 발생과 개발 단계의 지연을 방지한다.

2. 주요 configuration 분석

(1) Processor 파이프라인 파라미터

	분석	사이즈가 작을 경우	사이즈가 클 경우
fetch: ifqsize (default=4)	IFQ: IF와 ID 사이에 위치한 버퍼로, IF 단계에서 fetch된 명령어가 ID 단계로 넘어가기 전 잠시 저장.	IF 단계에서 넘어온 명령어를 저장할 공간이 부족해져 fetch 단계가 idle 상태가 될 확률이 높음.	지속적인 fetch가 가능하나 하드웨어의 낭비나 지나치게 많은 명령어를 fetch할 확률이 올라감.
decode: width (default=4)	ID에서 한 사이클에 동시에 처리할 수 있는 명령어의 최대 개수.	IFQ에 명령어가 쌓여 대기 시간 증가.	한 번에 많은 명령어 처리가 가능하나 IF나 EXE 단계가 받쳐주지 못하면 성능 항상 제한.
issue: width (default=4)	한 사이클에 EXE 단계로 전달할 수 있는 명령어의 최대 개수.	EXE 단계로 넘어오는 명령어 수가 너무 적어 idle 상태가 될 확률이 높음.	한 번에 많은 명령어를 실행할 수 있으나 ID의 처리 능력에 영향을 받음.

(2) bpred: branch predictor type

분기 예측 방법을 설정하는 configuration이다. 종류: {not-taken, taken, perfect, bimod, 2lev, comb, 1~6bit, percept, gehl}

프로젝트 내 사용	예측 방식	특징
not-taken	모든 분기가 조건을 만족하지 않는다고 가정.	예) for(i=0; i<5; i++)의 경우 i가 0~4일 때는 조건 만족(taken) → 예측 실패. 루프에서는 불리하고, if문 등 조건절에서는 조건 충족 빈도에 따라 유·불리가 갈린다.
taken	모든 분기가 조건을 만족한다고 가정.	예) for(i=0; i<5; i++)의 경우 i가 0~4일 때 조건 만족(taken) → 예측 성공. 루프에서는 유리하고, if문 등 조건절에서는 조건 충족 빈도에 따라 유·불리가 갈린다.
Bimod (default)	간단한 히스토리 기반 예측 방식 중 하나로 고정된 크기의 테이블('인덱스'라는 '셀'의 공간)을 이용한다. 테이블 크기(인덱스 개수)만큼 서로 다른 명령어를 저장할 수 있으며, 명령어의 인덱스에 따라 테이블에 매핑된다.	예) 명령어 1,2,3이 모두 인덱스 0을 쓰고 각각 11(Strongly taken), 01(Weakly not-taken), 10(Weakly taken)이라면, 인덱스 0은 계속 덮어써지며 값이 바뀐다(히스토리 고려 X). 인덱스가 겹치지 않으면 이전 결과가 남아있어 '간단한 히스토리'라고 부른다.

프로젝트 내 사용	예측 방식	특징
1 bit	직전 분기 결과를 1비트로 기억하여 다음 분기를 예측한다.	예) for(i=0; i<5; i++)에서 처음 i=0이므로 taken=0으로 설정, i<5까지는 계속 예측 성공. 마지막 i==5에서 예측 실패로 not-taken(1)로 변경 후 루프 종료.
3 bit	0~7의 총 8개 상태가 있으며, 값이 높을 수록 taken, 낮을수록 not-taken으로 예측한다.	예) 임계값 4(100)로 정한 경우 결과가 이상이면 taken, 이하면 not-taken. 한 번의 실패로 1bit 방식처럼 상태가 급변하지 않아 분기 조건이 복잡한 경우 적합하다. (-) H/W 자원 소모와 메모리 요구.
percept	입력 데이터와 가중치(weight)의 선형 결합을 계산해 결과를 예측한다. '활성화 함수'를 통해 출력이 특정 값을 넘으면 taken, 아니면 not-taken으로 판단한다.	Perceptron: 머신러닝에서 쓰이는 간단한 선형 분류기. (+) 효과적인 히스토리 활용, 높은 예측 정확도 (-) 복잡성, 긴 소요시간, 비선형적 분기 학습 제한.

(3) cache

	의미	목적
dl1	L1 데이터캐시 (CPU에 가장 가까운 캐시레벨)	데이터를 가장 빠르게 제공하여 속도를 높이는 목적 (데이터, 블록, 인덱스로 구성). {size, block size, associativity, replacement policy} 등을 조정 가능.
dl2	L2 데이터캐시	L1 cache보다 하위 레벨의 cache (L1 cache miss 보완). 전체적인 개요는 L1과 비슷함.
il1	L1 명령어 캐시	명령어를 저장하는데 사용하며 CPU에 가장 가깝게 위치한다. 명령어를 최대한 빠르게 전달하여 수행 속도를 높이는 목적. Fetch - cache hit/miss의 작동 방식이며 관련 파라미터는 dl1과 동일.
il2	L2 명령어 캐시	dl2와 동일.

(4) res

	분석
ialu	사용 가능한 정수 기반 ALU의 개수 (default=4). 명령어에 따라 산술(덧셈, 뺄셈, 논리) 연산 수행. 곱셈/나눗셈은 별도 유닛에서 처리.
imult	사용 가능한 곱셈 처리 ALU의 개수 (default=1). ialu에서 간단한 연산을 처리할 동안 imult에서는 복잡한 연산을 처리.

	분석
fpalu	사용 가능한 부동 소수점 산술 연산 유닛 개수 (default=4). integer기반 ALU에서 처리 못하는 소수 덧셈을 담당.
fpmult	사용 가능한 부동 소수점 곱셈 연산 유닛 개수 (default=1). integer기반 ALU에서 처리 못하는 소수 곱셈을 담당.

3. Processor 변화에 따른 성능 분석

(1) 연산 장치 수에 따른 성능 분석

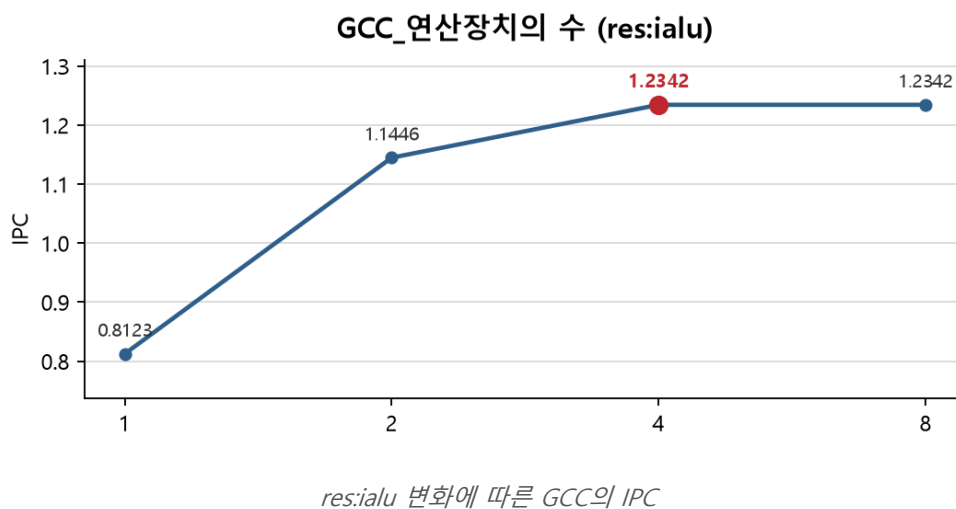
1. 연산장치 수에 영향을 미치는 configuration

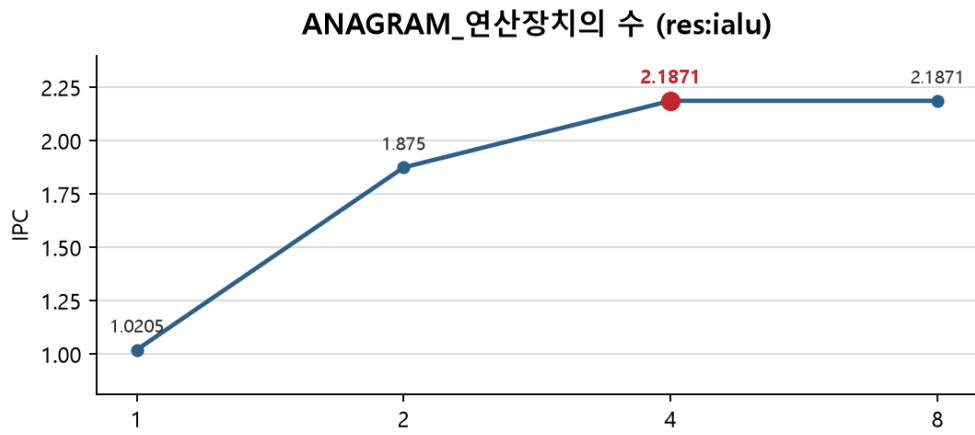
- `res:ialu` — 사용 가능한 정수 기반 ALU 의 개수에 대한 configuration 으로, GCC·ANAGRAM·COMPRESS95 는 정수 중심의 연산이 많아 integer ALU 의 수를 수정해 성능 개선을 평가하기 적합하다.
- `res:fpalu` — 사용 가능한 부동 소수점 산술 연산 유닛 개수에 대한 configuration 으로, GO 는 부동소수점 중심의 연산이 많아 floating point ALU 의 수를 수정해 성능 개선을 평가하기 적합하다.

2. 성능 비교를 위해 선택한 Simulation Statistics

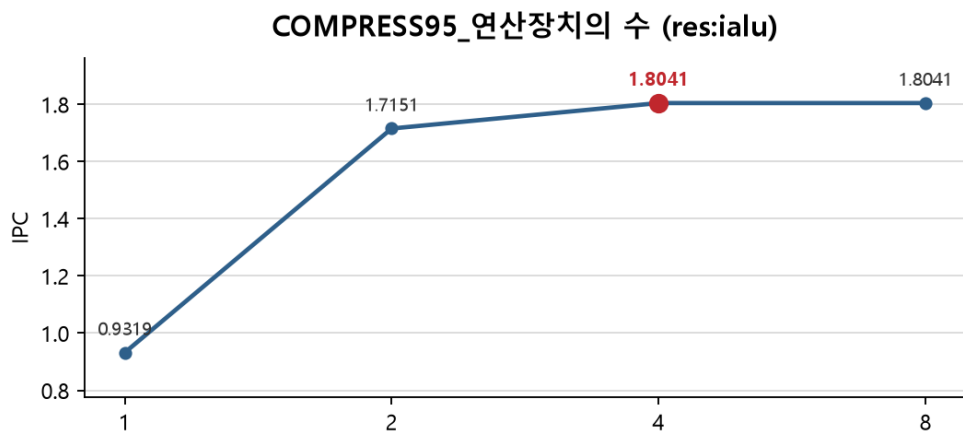
`sim_IPC` — 한 사이클에 처리되는 명령어 수로, 연산 장치 수 증가는 동시에 처리할 수 있는 명령어의 수가 늘어난다는 의미이다. IPC는 CPU가 얼마나 많은 명령어를 처리했는지를 보여주는 지표로 연산 장치 수 증가에 따른 성능 개선을 확인할 수 있는 지표이다.

3. 실행 결과 그래프

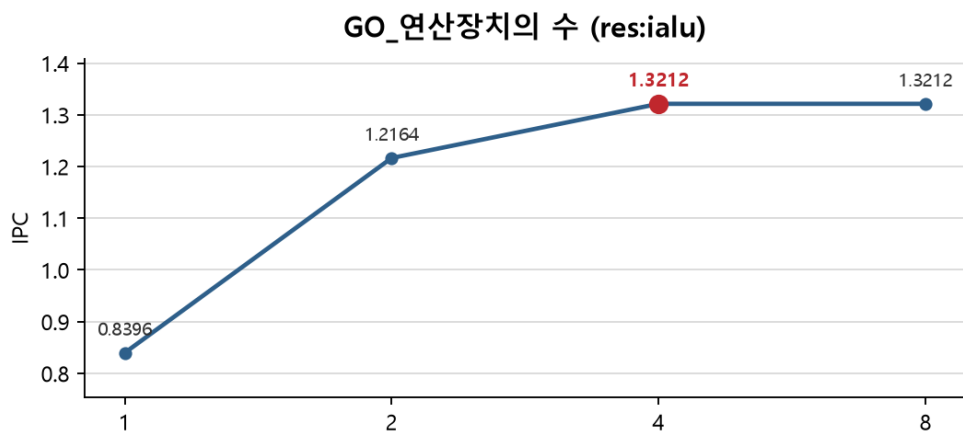




res:ialu 변화에 따른 ANAGRAM의 IPC



res:ialu 변화에 따른 COMPRESS95의 IPC



res:ialu 변화에 따른 GO의 IPC

4. 결과 분석

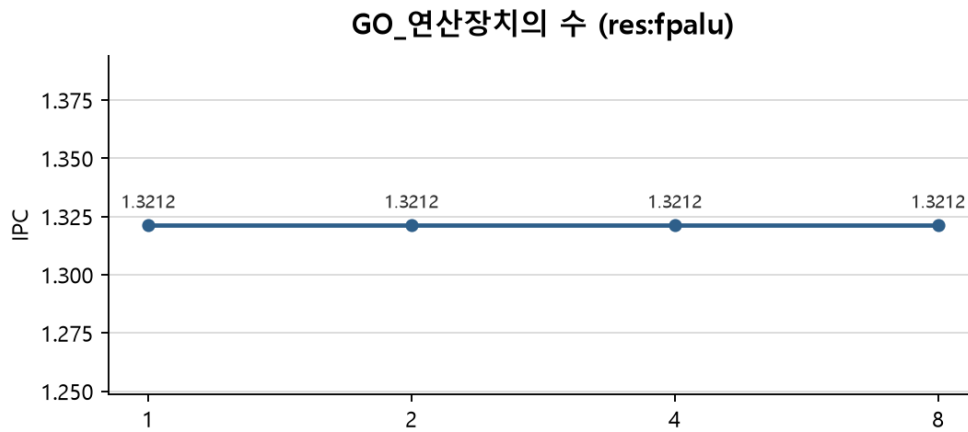
결과

1. 연산 장치 수가 증가함에 따라 IPC가 증가함
2. 연산 장치 수를 4→8개로 늘리면 IPC는 동일한 값

이유	1. 한 클럭 사이클 동안 병렬로 처리할 수 있는 명령어의 개수가 늘어났기 때문에 성능이 좋아짐 2. issue:width(한 사이클 동안 처리할 수 있는 명령어의 개수)가 4개로 고정돼, 연산 장치의 실제 활용이 제한된 것
단점	1. 하드웨어 낭비: 추가한 연산 장치가 활용되지 않는 경우 발생 2. 전력 소모 증가 3. 설계 비용 상승: 연산 장치 수 증가에 따른 설계 복잡도 상승

5. trouble shooting

프로그램 GO의 fpalu 개수를 변화시켰는데, IPC를 비롯해 다른 성능 지표들의 변화가 없었다. 프로그램 전체 명령어 중에서 부동소수점 연산이 차지하는 비율보다 정수 연산의 비율이 높아 ialu의 사용이 더 많은 것으로 판단해 ialu의 개수를 변화시켰다.



res:fpalu만 변화시켰을 때 GO의 IPC (변화 없음 — ialu 의존도가 높음을 시사)

(2) Super-scalar 에서 scalar 수에 따른 성능 분석

1. scalar 의 수에 영향을 미치는 configuration

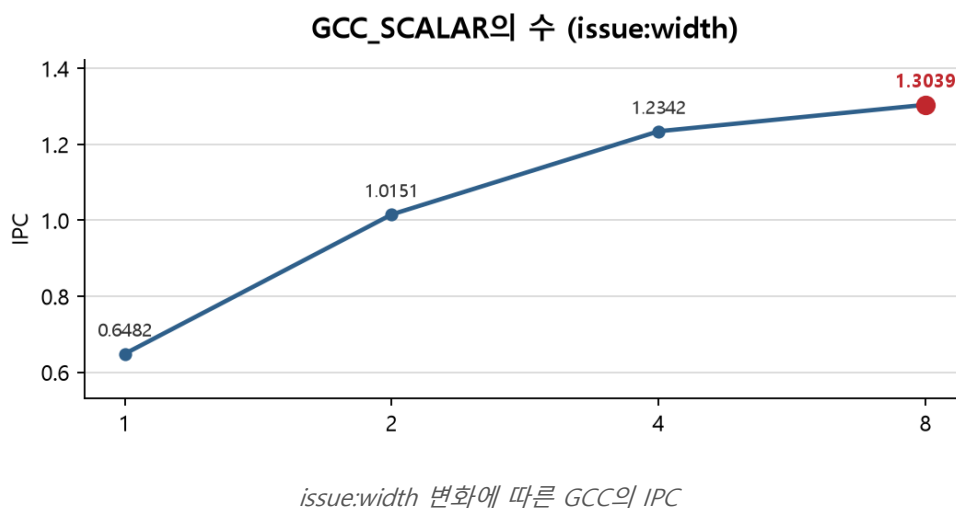
- `fetch:ifqsize` — IF 와 ID 사이 버퍼 사이즈. 값을 증가시키면 한 번에 가져올 수 있는 명령어 수가 증가해 CPU 명령어 처리 병목을 줄일 수 있다.
- `decode:width` — 한 사이클 동안 ID 에서 처리할 수 있는 명령어의 최대 개수. 값을 증가시키면 한 번에 처리할 수 있는 명령어의 개수가 늘어난다.
- `issue:width` — 한 사이클 동안 EXE 단계로 전달할 수 있는 명령어의 최대 개수. 값을 증가시키면 많은 명령어를 병렬 실행할 수 있다.

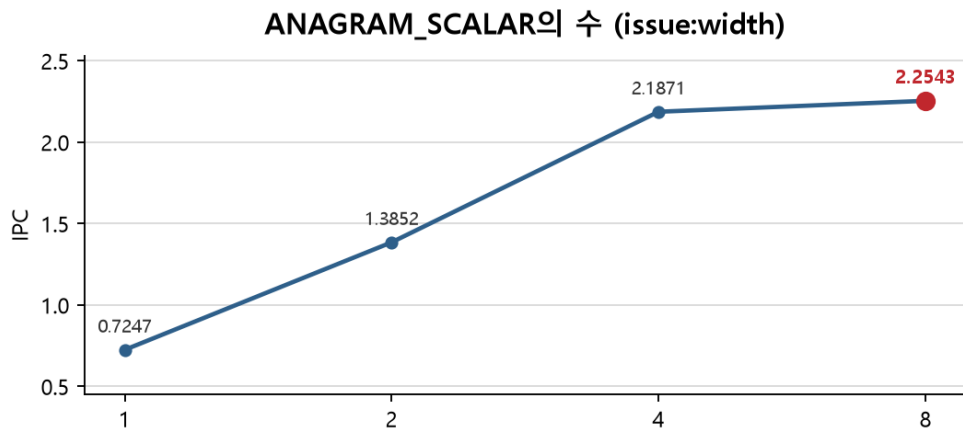
2. 성능 비교를 위해 선택한 Simulation Statistics

`sim_IPC` — IPC의 증가는 프로세서의 병렬 처리 효율 향상을 의미한다. 여러 명령어를 동시에 가져오고 decode하고, 발행 및 실행할 수 있는 병렬 처리 구조를 가진 super scalar 프로세서의 성능을 나타내기 적합하다.

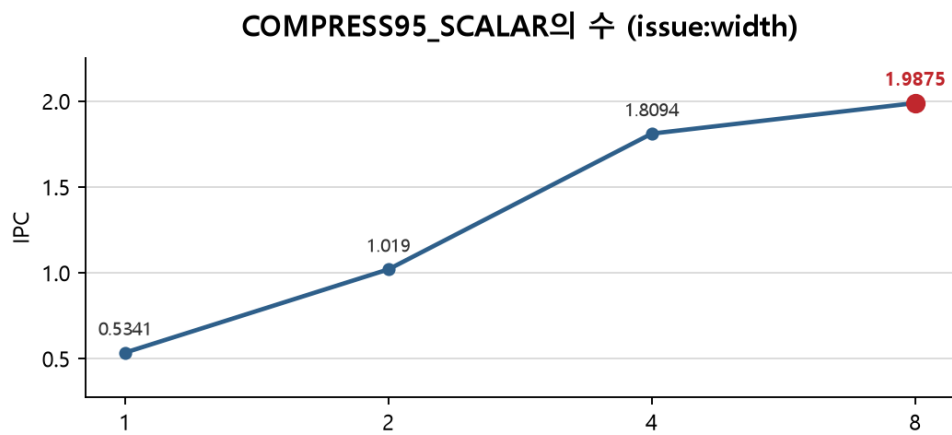
3. 실행 결과 그래프

프로젝트 내의 최대 스칼라 수에 맞추어 `fetch:ifqsize`, `decode:width`를 8로 고정했다. 이는 성능 분석의 다른 변수의 영향을 최소화하고 `issue:width`를 단독 변수로 설정해, 이의 변화가 성능에 미치는 영향을 분석하기 위함이다.

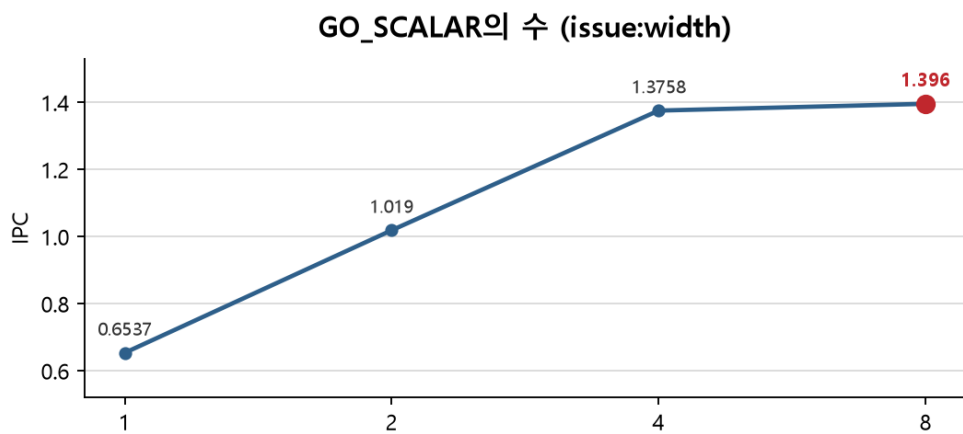




issue:width 변화에 따른 ANAGRAM의 IPC



issue:width 변화에 따른 COMPRESS95의 IPC

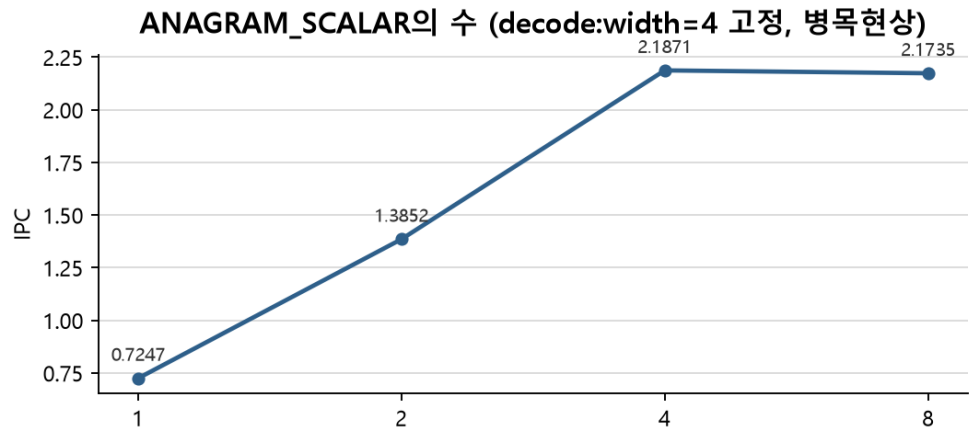


issue:width 변화에 따른 GO의 IPC

4. 결과 분석

결과	1. 스칼라 수가 증가함에 따라 IPC가 증가함
이유	1. 한 클럭 사이클 동안 병렬로 처리할 수 있는 명령어의 개수가 늘어났기 때문에

	성능이 좋아짐
단점	1. hazard 발생 증가 2. 병목현상 발생 가능성: 연산 장치의 부족, 데이터 의존성, ILP의 한계 등으로 병목 현상 발생 시 실제 성능 저하 3. 전력 소모 증가 4. 설계 비용 상승: 최적화를 위한 설계 복잡도 상승



decode:width가 4로 고정된 상태에서 scalar 수가 늘어도 IPC가 낮아지는 병목현상 (ANAGRAM)

(3) Branch predictor 종류에 따른 성능 분석

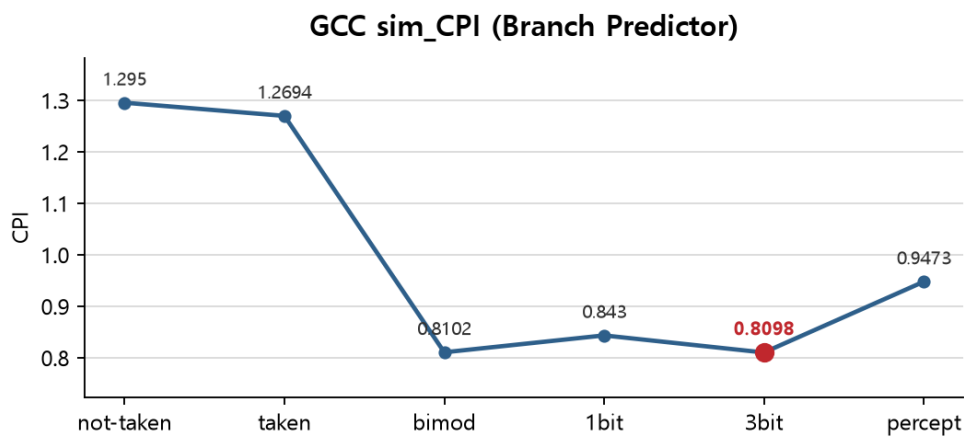
1. bpred 방식에 따른 결과

시뮬레이션 결과 `sim_num_insn`, `sim_num_loads`, `sim_num_branches`는 4가지 벤치마크마다 다르지만 bpred 방식에 따라 변하지는 않는다. `sim_cycle`, `sim_IPC`, `sim_CPI`는 bpred 방식마다 다른 결과를 보인다.

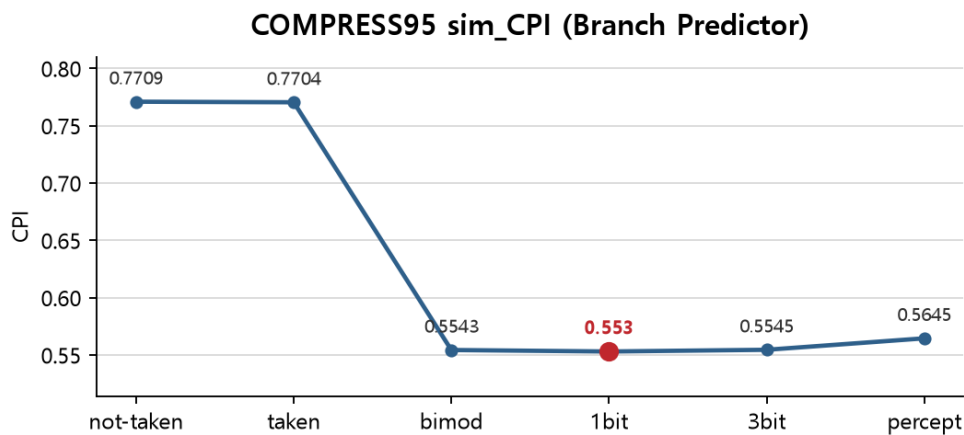
2. 성능 비교를 위해 선택한 Simulation Statistics

`sim_CPI` — 분기 명령어당 걸리는 사이클의 수를 직접적으로 파악할 수 있다. 예측 실패 시 flush가 발생하고 그에 따라 stall이 발생한다. 결과적으로 명령어 하나당 실행하는 사이클의 수가 늘어나기 때문에 CPI를 선택했다.

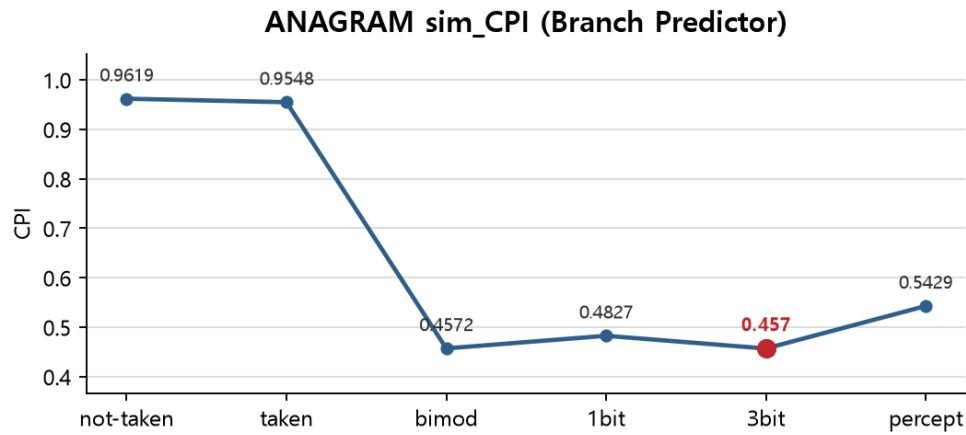
3. 실행 결과 그래프



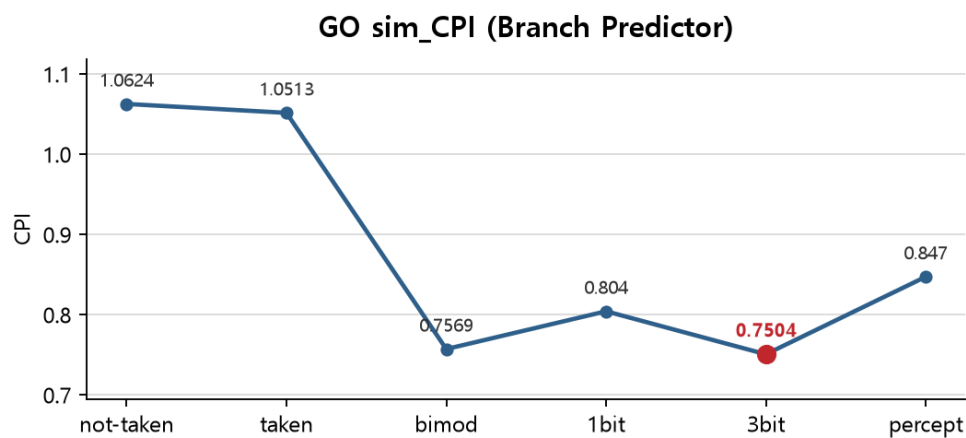
GCC sim_CPI (branch predictor 종류별)



COMPRESS95 sim_CPI (branch predictor 종류별)



ANAGRAM sim_CPI (branch predictor 종류별)



GO sim_CPI (branch predictor 종류별)

4. 결과 분석

(1) GCC / ANAGRAM / GO — 3-bit 이 default(bimod)보다 우수

결과	GCC, ANAGRAM, GO는 Default인 bimod보다 3-bit 측정 방식이 더 좋은 성능을 보임
이유	bimod: 2-bit를 이용해 연산 진행 → 예측 결과가 급격하게 변하는 현상(오버슈팅)이 발생할 가능성이 높음 3bit: 8가지 상태로 구성돼, 반복적인 전환이 아닌 경향성 있는 패턴에 강점. 중첩 반복문, 복잡한 조건문에서 bimod에 비해 안정적인 동작 가능 → 4개의 application은 일정한 경향성을 띠고 있으나, 조건과 반복문의 조합이 복잡해 1,2bit로 예측하기에는 상태 변화가 잦음
단점	1. 메모리 사용 증가: 8가지 상태를 저장해야 함 2. 테이블 크기가 일정할 경우 충돌 발생 가능성 증가(집 크기는 정해져 있는데 방 하나의 크기가 커지면 집 안의 방 개수가 줄어드는 것)

	3. 설계 비용 증가
	4. 분기 패턴이 급격하거나 일정한 경향성을 띠지 않는 경우 성능 저하

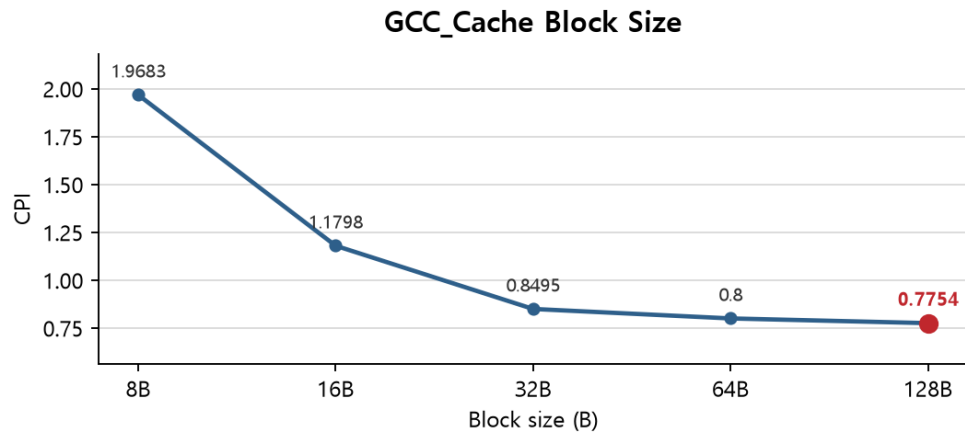
(2) COMPRESS95 — 1-bit 이 default(bimod)보다 우수

결과	COMPRESS95는 bimod보다 1-bit가 좋은 성능을 보임
이유	COMPRESS95는 분기 패턴이 단순하고 급격한 변화가 거의 없을 것으로 예측된다.
단점	분기 패턴이 자주 바뀌거나 복잡해질 경우 성능 저하 가능성이 큼

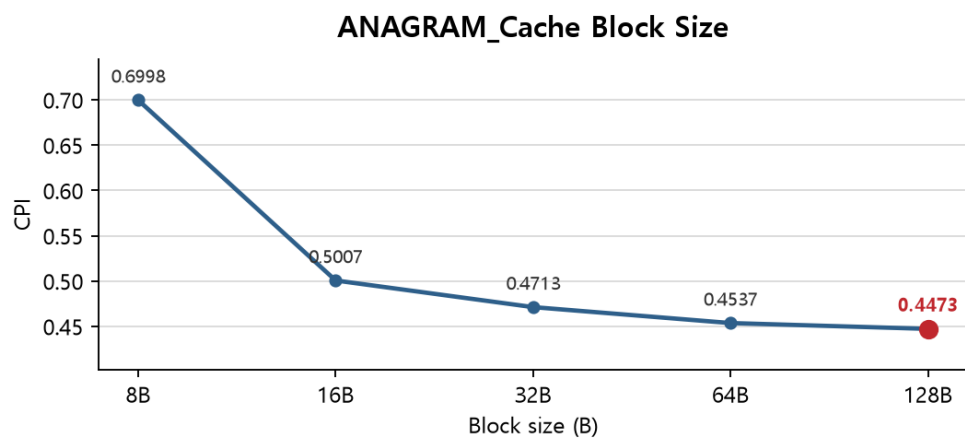
4. Cache 변화에 따른 성능 분석

(1) Cache block size 크기에 따른 성능 분석

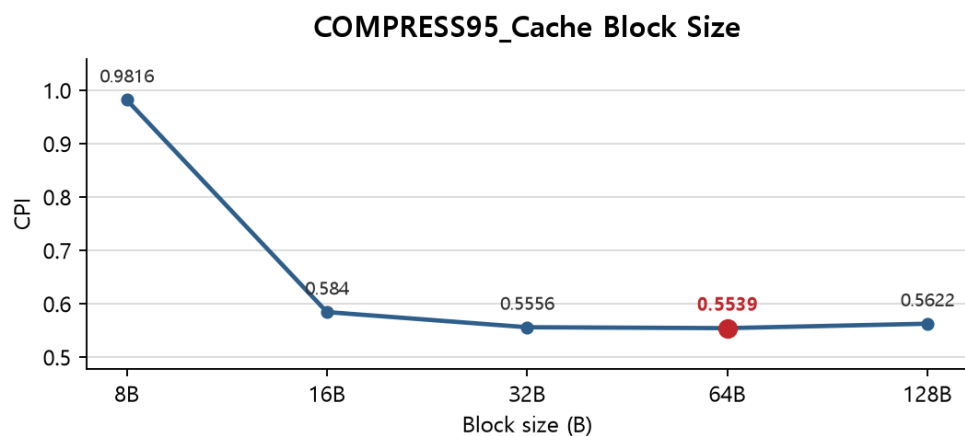
1. 실행 결과 그래프



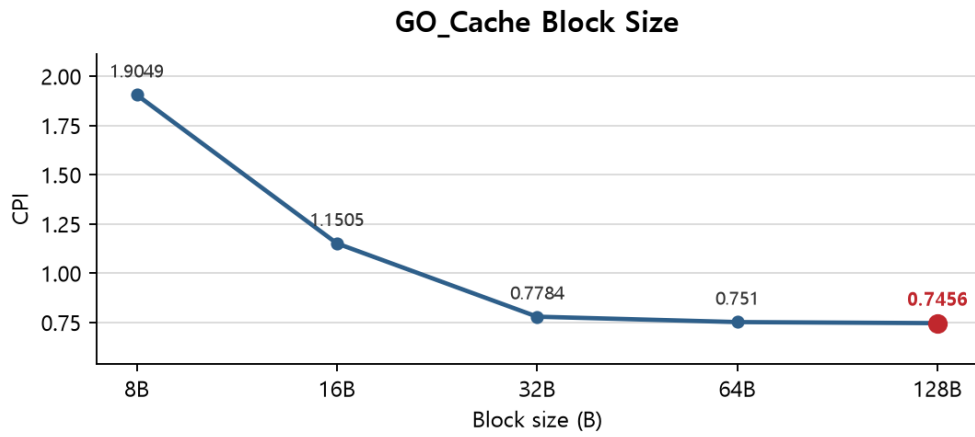
Block size 변화에 따른 GCC의 CPI



Block size 변화에 따른 ANAGRAM의 CPI



Block size 변화에 따른 COMPRESS95의 CPI



Block size 변화에 따른 GO의 CPI

측정 데이터

	block size	sim_cycle	IPC	CPI
gcc	8B	664027521	0.5080	1.9683
gcc	16B	390823223	0.8476	1.1798
gcc	32B	286580284	1.1772	0.8495
gcc	64B	269876171	1.2500	0.8000
gcc	128B	261586799	1.2896	0.7754
	block size	sim_cycle	IPC	CPI
anagram	8B	179152289	1.429	0.6998
anagram	16B	12817780	1.997	0.5007
anagram	32B	12064495	2.1217	0.4713
anagram	64B	11614446	2.2039	0.4537
anagram	128B	11448880	2.2358	0.4473
	block size	sim_cycle	IPC	CPI
compress95	8B	86573	1.0187	0.9816
compress95	16B	51510	1.7122	0.5840
compress95	32B	49004	1.7998	0.5556
compress95	64B	48851	1.8054	0.5539
compress95	128B	49584	1.7787	0.5622
	block size	sim_cycle	IPC	CPI

	block size	sim_cycle	IPC	CPI
go	8B	1039701598	0.525	1.9049
go	16B	627933043	0.8692	1.1505
go	32B	424864416	1.2847	0.7784
go	64B	409886062	1.3316	0.7510
go	128B	406961133	1.3412	0.7456

2. 결과 분석

결과	Block size가 증가함에 따라 총 cycle 수 감소, CPI 감소, IPC 값 증가 → 성능 증가
이유	1. Hit rate 증가: 큰 Block은 한 번의 캐시 미스에서 더 많은 데이터를 로드하므로 공간 지역성이 좋은 프로그램에서는 재사용될 확률이 높아짐 (예: 8B는 8바이트만, 64B는 64바이트를 한꺼번에 로드해 인접 데이터 재사용 증가) → 캐시 미스가 줄고 메모리 접근 횟수가 감소해 총 cycle 및 CPI 감소 2. 메모리 접근 오버헤드 감소: 큰 Block size를 통해 한 번의 메모리 접근에서 많은 데이터를 가져옴 3. 데이터 접근 지연이 줄어 CPU가 효율적으로 명령어를 실행할 수 있게 되므로 IPC 증가함
단점	1. 캐시 낭비: 큰 block size는 필요 없는 데이터까지 함께 로드해 공간 지역성이 낮은 프로그램에서는 캐시 낭비가 발생함 2. 캐시 메모리 낭비: 적은 수의 큰 Block이 캐시에 저장되므로 캐시 용량이 빠르게 차 conflict miss 확률이 높아짐 3. 메모리 대역폭 사용량 증가: 큰 Block을 자주 로드할 경우 발생할 수 있음

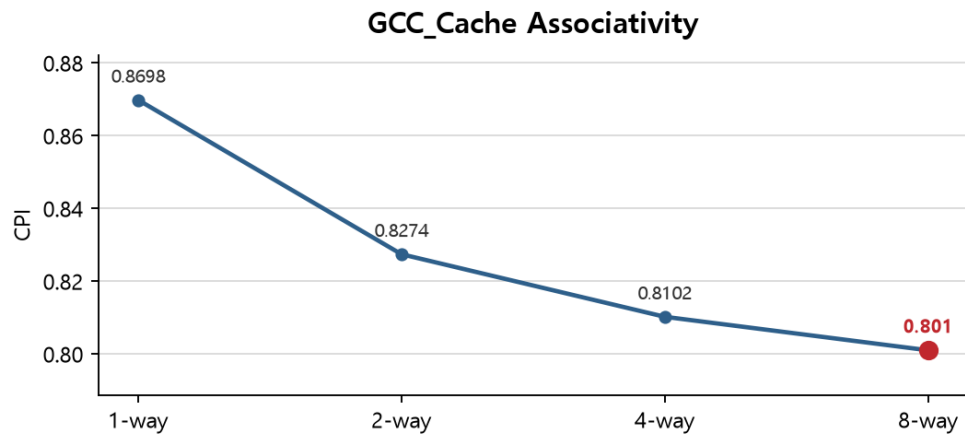
3. Trouble shooting

4-1 page를 수행할 때 처음에 Cache block size를 d1이 8B, 16B, 32B, 64B, 128B인 경우와 d2이 8B, 16B, 32B, 64B, 128B인 경우 총 10가지 옵션을 gcc, anagram, compress95 3개 어플리케이션에서 실행해 보았다. 하지만 d1의 Block size가 128B인 경우와 d2의 Block size가 8B, 16B인 경우 3개 어플리케이션이 모두 sim_cycle 값이 37, 35, 35로 오류가 나는 것을 확인했다.

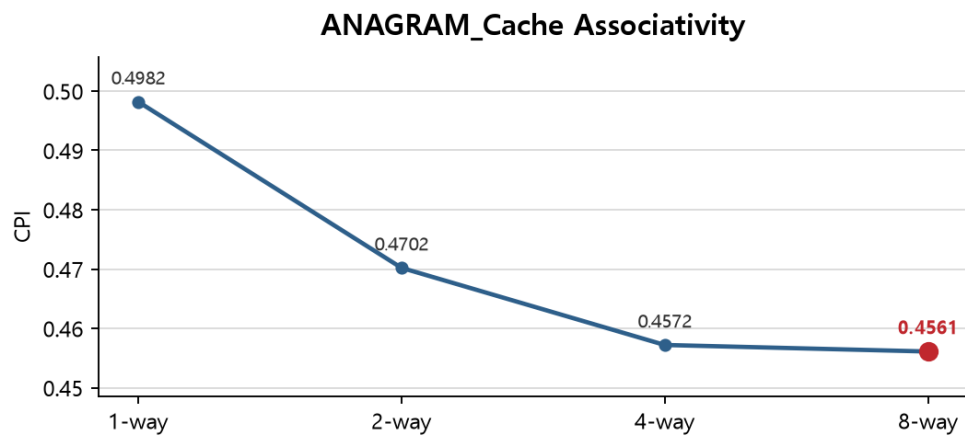
이는 두 cache 모두 8B, 16B, 32B, 64B, 128B의 경우로 하도록 통일하라는 것으로 잘못 이해해 발생한 문제로, 익명 게시판에 L1 cache block size를 8B로 수정할 때 L2 cache block size도 8B로 수정해야 하는지 확인 차 질문드렸다. 답변("L1/L2 cache를 모두 같이 바꾸라는 뜻입니다")을 받아 4-1 page 프로그램을 수정해 다시 수행했다.

(2) Cache associativity 에 따른 성능 분석

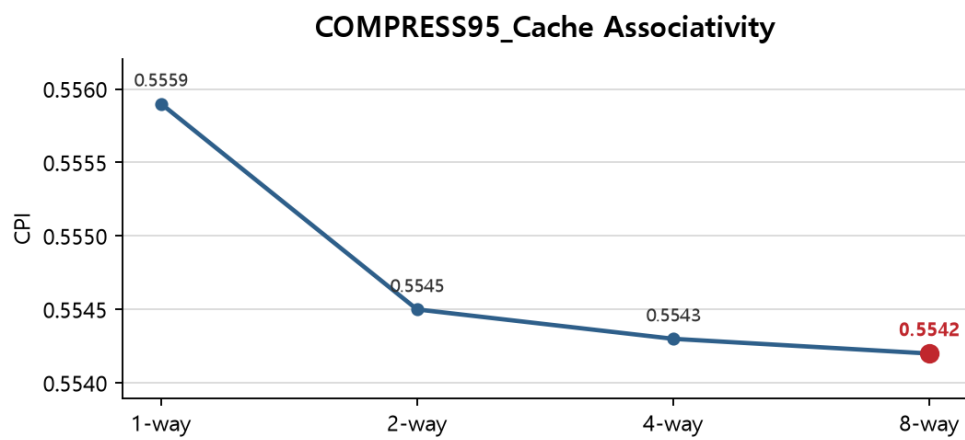
1. 실행 결과 그래프



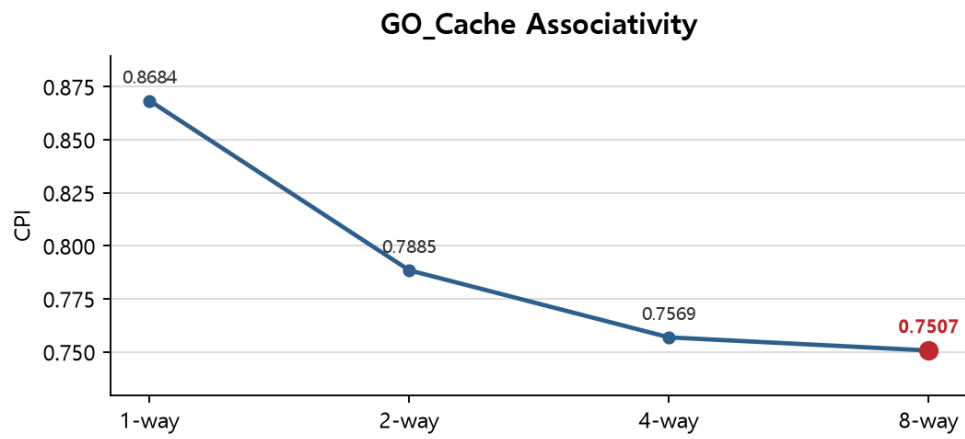
Associativity 변화에 따른 GCC의 CPI



Associativity 변화에 따른 ANAGRAM의 CPI



Associativity 변화에 따른 COMPRESS95의 CPI



Associativity 변화에 따른 GO의 CPI

측정 데이터

	associativity	sim_cycle	IPC	CPI
gcc	1	293432840	1.1497	0.8698
gcc	2	279129843	1.2086	0.8274
gcc	4	273328572	1.2342	0.8102
gcc	8	270216296	1.2485	0.8010
	associativity	sim_cycle	IPC	CPI
anagram	1	12753000	2.0072	0.4982
anagram	2	12034898	2.1269	0.4702
anagram	4	11703806	2.1871	0.4572
anagram	8	11676235	2.1923	0.4561
	associativity	sim_cycle	IPC	CPI
compress95	1	49026	1.7989	0.5559
compress95	2	48902	1.8035	0.5545
compress95	4	48885	1.8041	0.5543
compress95	8	48877	1.8044	0.5542
	associativity	sim_cycle	IPC	CPI
go	1	473985365	1.1515	0.8684
go	2	430372298	1.2682	0.7885
go	4	413103518	1.3212	0.7569

	associativity	sim_cycle	IPC	CPI
go	8	409742108	1.3321	0.7507

2. 결과 분석

결과	Cache associativity가 증가함에 따라 총 cycle 수 감소, CPI 감소, IPC 값 증가 → 성능 증가
이유	1. Hit rate 증가: 특정 set 내에 더 많은 cache lines를 저장할 수 있어 동일 set에 매핑된 여러 메모리 주소가 캐시에 저장될 확률이 높아져 충돌로 인한 캐시 미스가 감소 → pipeline stall이 줄어 IPC 증가 → CPI 감소 2. 캐시 충돌 감소: 1-way(Direct-mapped)는 특정 주소가 반드시 특정 line에만 매핑되어 충돌 시 기존 데이터를 덮어써야 하지만, 2/4/8-way(Set-associative)는 선택 가능한 line이 늘어나 충돌이 줄고 miss가 감소함 3. 메모리 접근 시간 감소: 캐시 히트율이 높아지면 메인 메모리 접근 횟수가 줄어들어(전체 실행 시간 단축)
단점	1. 탐색 시간(latency) 증가: associativity가 높아질수록 set 안의 모든 cache line을 동시에 검사해야 함 2. HW의 복잡성 증가: tag 비교나 replacement policy 구현이 어려워짐(LRU는 associativity가 높아질수록 더 많은 state 관리 필요) 3. 전력 소모 증가: 더 많은 cache line과 비교 회로 필요 4. Cache 접근 속도 저하: 병렬 비교가 많아져 캐시 접근 시간이 늘어날 수 있음

(3) Cache replacement policy 에 따른 성능 분석

1. cache replacement policy

페이징 기법으로 메모리를 관리하는 운영체제에서 페이지 부재가 발생해 새로운 페이지를 할당하기 위해 현재 할당된 페이지 중 어느 것과 교체할지 결정하는 방법이다.

	의미	특징
l	LRU (Least Recently Used)	가장 오랫동안 사용하지 않은 캐시 블록 교체. 장점) 일반적으로 캐시 미스율이 낮아 성능이 좋음. 단점) 1. 사용 횟수 추적에 따른 오버헤드 발생 2. 하드웨어 비용 증가(구현 복잡도가 높고 각 블록의 사용 시점 추적 필요)
f	FIFO (First In, First Out)	가장 먼저 캐시에 들어온 데이터를 교체(사용 여부 고려 안 함, time stamp 필요). 장점) 구현이 단순하고 하드웨어 요구 사항 적음. 단점) 높은 miss rate 가능성(가장 오래된 캐시가 가장 많이 쓰는 데이터일 수도 있음)
r	RANDOM	캐시에 저장된 블록 중 하나를 임의의 선택해 교체. 장점) 구현이 매우 간단하고 특정 패턴(순환접근)에 대응 용이. 단점) 성능 불안정(교체 결정이 비효율적)

이론에 따른 simulation 예측

- LRU-LRU: 데이터 지역성을 활용하여 L1 과 L2 에서 Miss Rate 를 낮출 것으로 예상된다.
- FIFO-FIFO: 오래된 데이터를 교체하므로 지역성을 활용하지 못해 Miss Rate 가 증가할 것으로 예상된다.
- RANDOM-RANDOM: 무작위 교체로 인해 Miss Rate 와 AMAT 가 가장 높을 것으로 예상된다.

2. 성능 비교 시 주요 configuration 선택

(1) dl1, dl2 miss rate

L1 데이터 캐시와 L2 데이터 캐시에서 데이터 접근 실패가 발생하는 비율.

- 1. 캐시 설계의 효과를 직접적으로 반영: 교체 정책, 캐시 크기, 블록 크기 등의 변경이 캐시 미스율에 직접적인 영향을 미친다.
- 2. 미스율 감소는 캐시 성능 향상을 의미하며, 메모리 접근 시간 및 CPI 개선으로 이어진다.
- 3. 메모리 계층 구조의 효율성을 평가: dl1 미스율은 L1 캐시의 효율성을, dl2 미스율은 L1 에서 해결되지 못한 데이터를 L2 에서 처리하는 효율성을 나타낸다.
- 4. 미스율은 AMAT 및 CPI 감소: 캐시 미스율을 낮추는 것은 메모리 대기 시간을 줄이는 핵심 요소이다.

(2) CPI

명령어 하나를 실행하는 데 소요되는 평균 사이클 수.

- 1. 전체 시스템 성능의 직관적 지표
- 2. 미스율과 CPU 병목 현상을 반영: 높은 미스율은 메모리 대기 시간을 증가시켜 CPI 에 부정적 영향을 미치므로, 캐시 미스율 변화가 CPI 에 어떻게 반영되는지 명확히 보여준다.
- 3. CPI 는 캐시뿐 아니라 메모리 계층의 전체적인 상호작용을 반영하므로, 최적의 캐시 구성이 CPI 개선에 얼마나 기여했는지 정량적으로 평가 가능하다.

성능 비교 방법 — AMAT (Average Memory Access Time)

dl1과 dl2의 미스율을 기반으로 평균 메모리 접근 시간(AMAT)을 계산하여 AMAT와 CPI를 기준 삼아 각 구성의 성능을 비교한다.

- dl1 miss rate: L1 캐시의 데이터 접근 실패 비율
- dl2 miss rate: L2 캐시에서 L1 미스를 처리하지 못한 비율
- L1 Hit Time: L1 에서 데이터를 찾았을 때의 시간 (1 사이클로 가정)
- L1 Miss Penalty: L1 미스 발생 시 L2 캐시에 접근하는 데 걸리는 시간 (L2 Hit Time, 10 사이클로 가정)
- L2 Miss Penalty: L2 캐시 미스 발생 시 메인 메모리 접근 시간 (100 사이클로 가정)

1. L2 Access Time = L1 Miss Penalty + (L2 Miss Rate × L2 Miss Penalty)

2. AMAT = L1 Hit Time + (L1 Miss Rate × L2 Access Time)

→ 정리하면: $AMAT = L1\ Hit\ Time + (L1\ Miss\ Rate \times (L1\ Miss\ Penalty + (L2\ Miss\ Rate \times L2\ Miss\ Penalty)))$

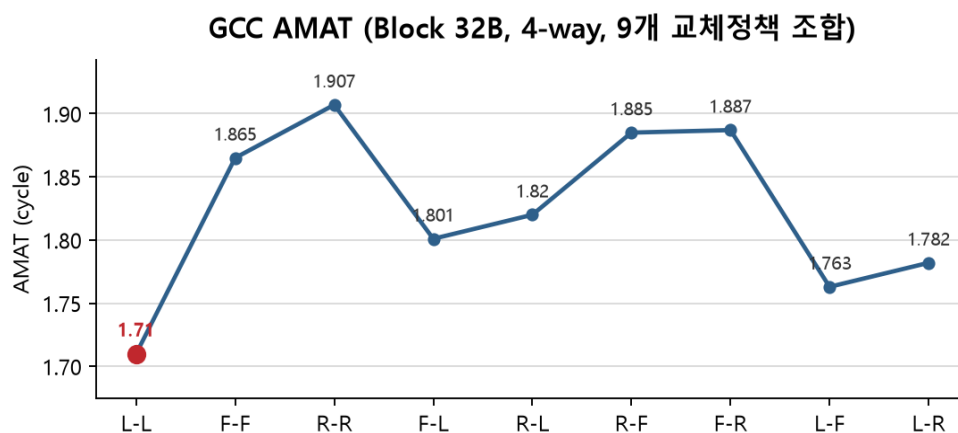
3. application 별 분석

Cache block size는 32B로 통일하고 Associativity=4-way, 데이터 캐시 크기는 d1=128, d2=1024로 고정한다. 이는 교체 정책에 따른 성능 차이만을 명확히 비교할 수 있도록 하기 위함이다. 경우의 수 9개로 simulation을 진행했으며, 각각에 따른 AMAT와 CPI를 비교한다.

실험 번호	L1 교체 정책	L2 교체 정책	조합
1	LRU (l)	LRU (l)	L-L
2	FIFO (f)	FIFO (f)	F-F
3	RANDOM (r)	RANDOM (r)	R-R
4	FIFO (f)	LRU (l)	F-L
5	RANDOM (r)	LRU (l)	R-L
6	RANDOM (r)	FIFO (f)	R-F
7	FIFO (f)	RANDOM (r)	F-R
8	LRU (l)	FIFO (f)	L-F
9	LRU (l)	RANDOM (r)	L-R

I. GCC

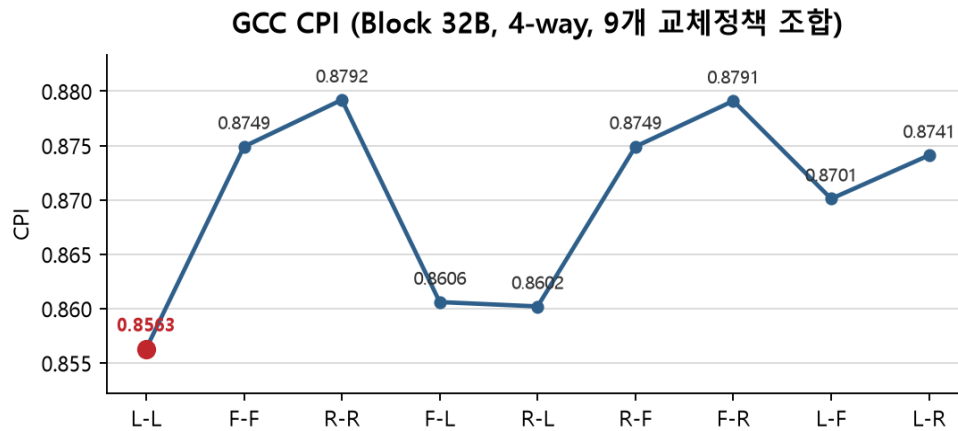
- AMAT



최소 AMAT	L-R 조합 RANDOM(L2)가 특정 워크로드(데이터 사용 패턴)에서 효율적으로 작동한 사례로 보임. (보통 RANDOM은 데이터를 무작위로 교체하므로 효율적이지 않을 것이라고 예상)
최대 AMAT	R-R 조합

RANDOM은 데이터를 무작위로 교체 → Miss Rate가 높게 나옴. 적중률이 크게 저하돼 Miss Rate 증가함.

- CPI



최소 CPI	L-L 조합 GCC 워크로드의 시간적 지역성과 공간적 지역성을 LRU 정책이 최대한 활용함 → 재사용 가능성이 낮은 데이터만 제거
최대 CPI	R-R 조합 RANDOM 조합은 데이터 지역성을 전혀 반영하지 않아 재사용 가능성이 높은 데이터도 제거됨 → L1과 L2 모두 데이터 접근 패턴과 맞지 않아, 비효율적인 결과가 나온 것으로 예상.

데이터 사용 패턴과 정책 간의 상호작용이 성능에 큰 영향을 미침을 알 수 있다. LRU+RANDOM처럼 상호 보완적인 정책은 의외로 높은 성능을 기록할 수 있다.

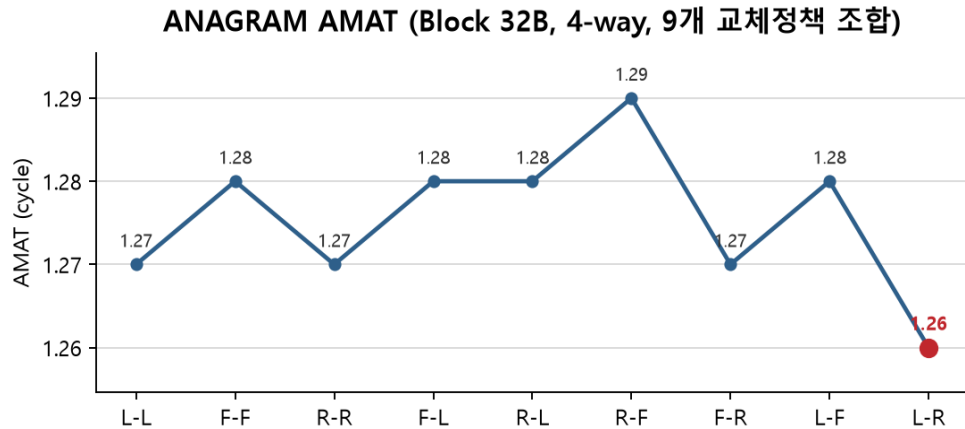
(L-L) 정책의 단점

AMAT와 CPI를 분석하였을 때, 둘 다 L-L 정책의 성능이 좋게 나타났다.

- (1) 구현 복잡성: 각 데이터의 사용 시간을 추적해야 하므로 하드웨어 비용이 증가한다.
- (2) 데이터 오버헤드: 교체할 데이터를 찾기 위해 전체 캐시 블록을 검색하거나 데이터 구조를 유지해야 한다. 캐시 크기가 클수록 이러한 작업이 병목으로 이어져 성능 저하가 일어날 가능성이 높다.

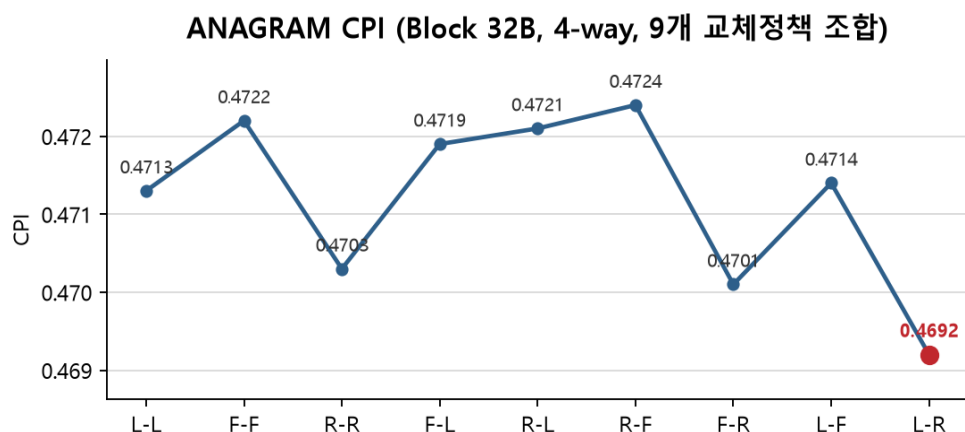
II. ANAGRAM

- AMAT



최소 AMAT	L-R 조합 L1에서 Miss Rate가 0.0047로 매우 작기 때문에 L2의 Miss Rate가 AMAT에 미치는 영향을 상당 부분 상쇄했을 가능성이 높음. 대부분의 데이터 접근이 L1에서 처리되었기 때문에 L2 정책이 비효율적이더라도 메모리 접근 횟수가 크게 증가하지 않게 됨.
최대 AMAT	R-F 조합 FIFO는 데이터를 단순히 들어온 순서대로 교체하기 때문에 재사용 가능성이 높은 데이터도 제거함. 1024라인이라는 큰 캐시에서는 FIFO가 데이터 재사용을 제대로 활용하지 못하여 적중률이 저하됨. (RANDOM: 작은 캐시에서 데이터 보존 어려움 / FIFO: 큰 캐시에서도 비효율적인 교체)

- CPI



최소 CPI	L-R 조합 128라인의 LRU는 작은 캐시 크기에서 데이터의 시간적 지역성을 효과적으로 활용. RANDOM의 단순한 동작이 캐시 오버헤드를 줄여 최적 성능이 나온 것으로 예상
---------------	--

최대 CPI	R-F 조합 R-F의 성능이 낮아 F-R 조합도 성능이 낮을 것이라 생각했는데 실제로는 높은 성능을 보임. 단순한 FIFO지만 1024라인 RANDOM과 결합하면 성능이 최적화되는 것을 알 수 있음
--------	---

L-F의 경우처럼 L1과 L2 정책이 서로를 보완할 경우 성능이 개선될 수 있다. R-R 정책과 같이 비효율적인 정책이 중첩되면 성능이 낮아질 수 있다.

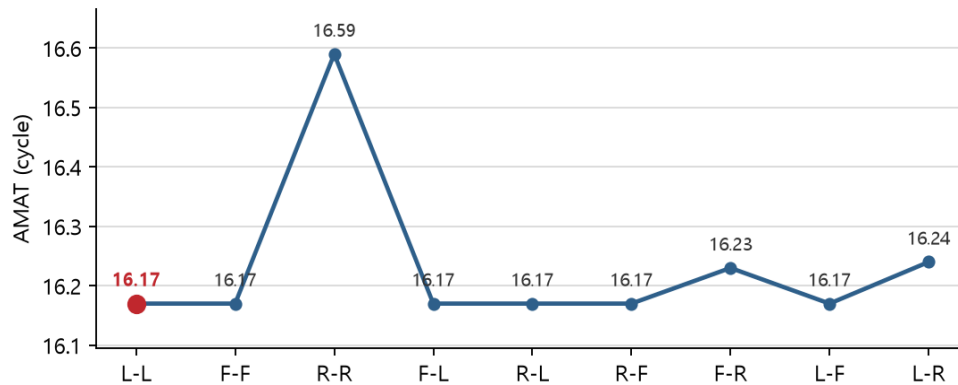
(L-R) 정책의 단점

RANDOM 정책은 1024라인 캐시에서 무작위로 데이터를 교체하므로 예측 가능성이 낮아 디버깅 및 분석이 어렵다. 큰 캐시에서는 비효율적 데이터 관리가 보완될 수 있지만, 캐시 크기를 더 줄이거나 워크로드가 변화하면 이 조합은 비효율적일 수 있다.

III. COMPRESS95

- AMAT

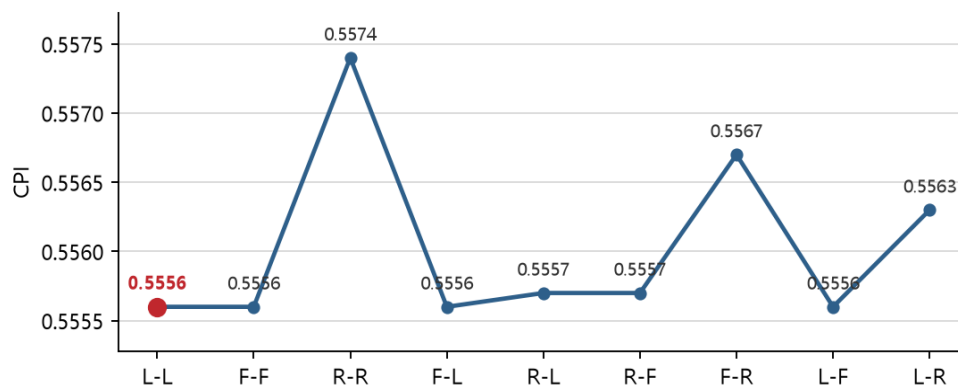
COMPRESS95 AMAT (Block 32B, 4-way, 9개 교체정책 조합)



최소 AMAT	L-L, F-F, F-L, R-L, R-F, L-F 조합 (동일한 최소값) 모두 특정 워크로드에서 효율적으로 작동해 메모리 접근을 최소화함
최대 AMAT	R-R 조합 L1 → 데이터를 무작위로 교체해 자주 사용되는 데이터를 캐시에 유지할 확률이 낮음. L2 → 무작위로 데이터를 교체해 L1에서 실패한 데이터가 L2에서도 실패할 가능성이 높음

- CPI

COMPRESS95 CPI (Block 32B, 4-way, 9개 교체정책 조합)



최소 CPI	다수의 조합 (L-L, F-F, F-L, L-F 등)에서 동일한 최소값 워크로드가 특정 정책(LRU와 FIFO)에서 안정적으로 작동함을 보여줌
최대 CPI	R-R 조합 RANDOM 정책의 무작위 교체로 인해 메모리 접근 횟수가 증가했을 가능성이 높음

특정 데이터 접근 패턴에 대한 정책 적합성의 중요성을 알 수 있다. 어플리케이션 별로 데이터 접근 패턴이 다르기 때문에, 모든 워크로드에 대해 동일한 최적 정책은 존재하지 않는다.

COMPRESS95 의 워크로드 특성

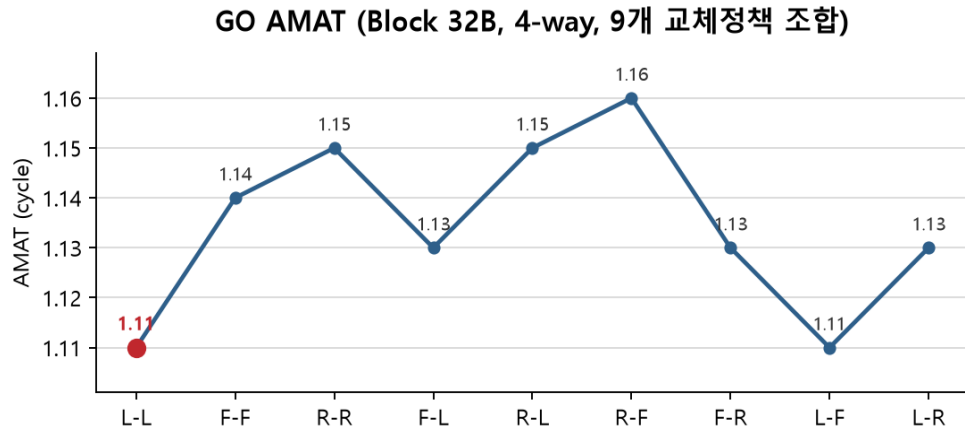
데이터 접근 패턴은 비교적 균일하거나, 특정 정책의 영향을 크게 받지 않는 특성을 보인다. L1(LRU 또는 FIFO)과 L2(LRU 또는 FIFO) 정책이 혼합된 경우에도 성능 차이가 거의 나타나지 않는다. 하지만 RANDOM(L2)은 데이터 접근 패턴과 맞지 않아 일부 조합(R-R, F-R, L-R)에서 성능이 저하된다.

(L-L, F-F) 정책의 단점

- (1) L-L 조합: LRU 정책이 데이터를 효율적으로 관리하지만, 캐시 자원이 중복 사용될 가능성이 있으며 데이터 추적과 관리 비용이 늘어날 수 있다.
- (2) F-F 조합: 지역성을 고려하지 않기 때문에 특정 워크로드에서 자주 사용하는 데이터를 불필요하게 교체해 성능이 저하될 위험이 있다.

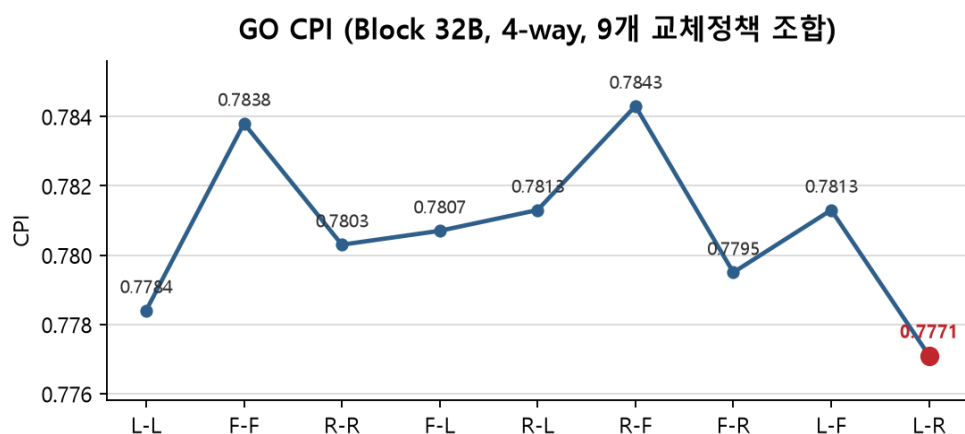
IV. GO

- AMAT



최소 AMAT	L-L, F-F 조합 충분한 캐시 공간 제공으로 LRU의 강점이 극대화됨. 1024라인에서는 더 많은 데이터를 저장할 수 있어 교체 빈도가 줄어 성능 향상에 기여했을 가능성이 크다. FIFO는 단순한 정책이지만 워크로드가 균일한 데이터 접근 패턴을 보였을 가능성이 높음.
최대 AMAT	R-F 조합 데이터 접근 특성을 고려하지 않는 정책으로 서로 보완적인 작용을 하지 못하고 비효율성을 더함. (RANDOM: 자주 사용되는 데이터까지 무작위로 제거 / FIFO: 순차적으로 데이터를 제거해 불필요한 캐시 미스 증가)

- CPI



최소 CPI	L-R 조합 L1 캐시의 높은 적중률이 RANDOM 정책의 비효율성으로 인한 CPI 상승을 억제함 → L2 캐시 및 메인 메모리 접근이 줄었을 것으로 예상할 수 있음
---------------	--

최대 CPI	R-F 조합 지역성을 반영하지 못함 → 자주 사용하는 데이터가 교체될 가능성이 높음
---------------	---

AMAT의 결과를 보면 R 정책을 포함한 R-R과 R-L 정책도 AMAT가 높게 나타난다. 정책 조합은 성능에 큰 영향을 미치므로 비효율적인 조합을 피해야 하며, 작은 캐시 크기에서는 비효율적인 정책의 영향이 더 크게 나타나므로 캐시 크기를 고려해 효율적인 교체 정책을 적용해야 한다.

(L-R) 정책 단점

캐시 계층 간 부하 불균형이 발생할 수 있다. L2 캐시가 L1 캐시의 저하를 충분히 보완하지 못한다. 이는 L1 캐시의 의존도가 강하다는 것을 의미하며 L1 캐시에서 miss가 발생할 경우 성능이 급격히 저하될 위험이 있다. 뿐만 아니라 워크로드 의존성도 강해, 워크로드가 조금만 변해도 성능 저하가 일어날 가능성이 크다.

(L-L) 정책 단점

데이터를 효율적으로 관리해 CPU와 메모리 간의 협력을 극대화했지만, 하드웨어를 구현하는 비용이 크고 캐시 중복 사용 가능성이 높아질 수 있다.

결론

특정 워크로드에서 최적화된 정책이라 하더라도, 모든 상황에서 동일한 성능을 보장하지 않으며, 정책의 선택은 워크로드의 특성과 하드웨어 구현 가능성, 자원을 종합적으로 고려해야 한다.

교체 정책에 따른 성능 파악을 더 정확히 하기 위해 AMAT와 CPI 두 개의 성능 지표를 사용했는데, 교체 정책을 단일 기준으로 평가하기에는 어려움을 가진다. 또한 정책 혼합의 중요성을 알 수 있다. L1과 L2의 정책 조합이 상호 보완적일 때 성능이 개선되는 것으로 예측할 수 있고, 특정 정책이 좋은 성능을 나타내는 것이 아닌 application의 특징과 데이터 접근 방식에 따라 좋은 정책 조합들이 달라짐을 알 수 있다.

5. 최적의 cache configuration 탐색

1. simulation 방법

simulation을 진행하기에 앞서 최적의 Block size와 Associativity를 선정했다. 최적의 Block size는 128B, Associativity는 8-way로 고정한 뒤, replacement policy를 L, F, R을 조합한 9가지 경우의 수로 나누어 진행했다. (단, COMPRESS95는 앞선 분석에서 64B가 최적이었으므로 64B를 기준으로 사용했다.)

simulation 방법 선택 이유

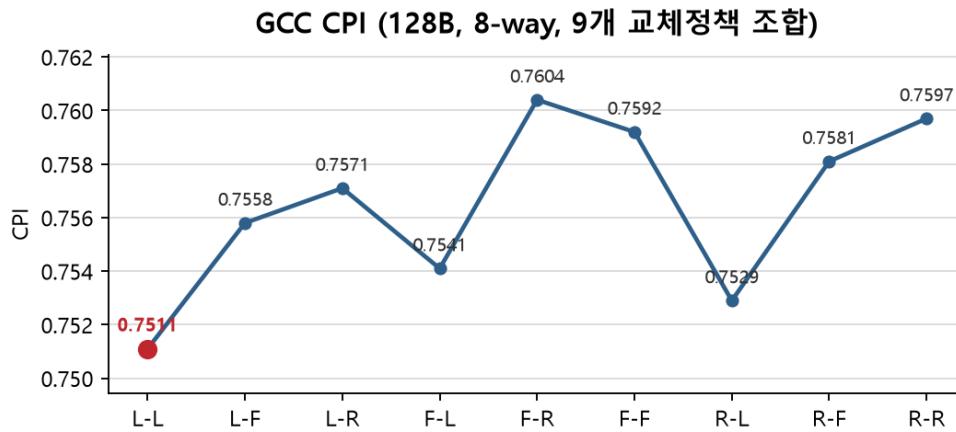
- (1) application 별 최적화 가능성 분석 가능: application 마다 데이터 접근 패턴이 다르므로 동일한 replacement policy 가 모든 application 에서 최적이지 않음을 앞선 4-(3) 분석을 통해 알 수 있다. 최적의 block size 와 associativity 를 고정한 상태에서 교체정책만 변경하면 application 특성에 따라 각기 최적 정책을 찾는데 용이하다. 4-(1), 4-(2) simulation 을 통해 대부분의 application 에서 block size 가 커지고 associativity 가 증가할수록 성능이 향상됨을 확인했으므로, 각각의 최적 상태를 고정해 9 가지 정책 조합만 바꿔가며 최적의 성능 조합을 찾기로 결정했다.
- (2) block size 와 associativity 의 작동 원리 차이: block size 는 한 번의 캐시 접근 시 읽거나 쓰는 데이터의 크기를 결정하는 파라미터이고, associativity 는 동일한 캐시 세트 내에서 데이터를 저장하는 방식과 유연성을 결정하는 파라미터이다. 두 파라미터는 서로 다른 캐시 구조의 동작 측면에서 성능에 기여하므로 이러한 작동 원리의 차이를 이용해 simulation 을 설계할 수 있다.
- (3) 실험의 효율성: 모든 경우의 수를 고려할 경우 총 144 번의 simulation 이 필요해, 캐시 성능에 독립적으로 큰 영향을 미치는 주요 파라미터(block size, associativity)를 먼저 고정해 simulation 횟수를 최소화했다.

2. simulation 결과 (캐시 크기: d1=128, d2=1024)

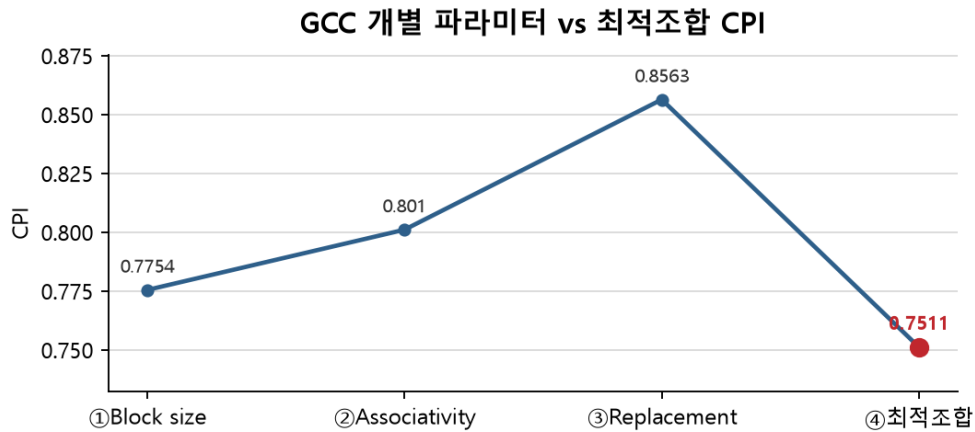
I. GCC

실험 번호	L1 교체 정책	L2 교체 정책	조합
1	LRU (l)	LRU (l)	L-L
2	LRU (l)	FIFO (f)	L-F
3	LRU (l)	RANDOM (r)	L-R
4	FIFO (f)	LRU (l)	F-L

실험 번호	L1 교체 정책	L2 교체 정책	조합
5	FIFO (f)	RANDOM (r)	F-R
6	FIFO (f)	FIFO (f)	F-F
7	RANDOM (r)	LRU (l)	R-L
8	RANDOM (r)	FIFO (f)	R-F
9	RANDOM (r)	RANDOM (r)	R-R



Cache block size 128B, Associativity 8-way 고정 시 GCC CPI — L-L 조합이 0.7511로 최소



①block size=128B	0.7754
②associativity=8-way	0.8010
③replacement(L-L)	0.8563
④최적화 조건(3가지 결합)	0.7511

세 가지를 최적 조합으로 실행했을 때의 CPI가 가장 낮기 때문에 최적화에 성공했음을 확인할 수 있다.

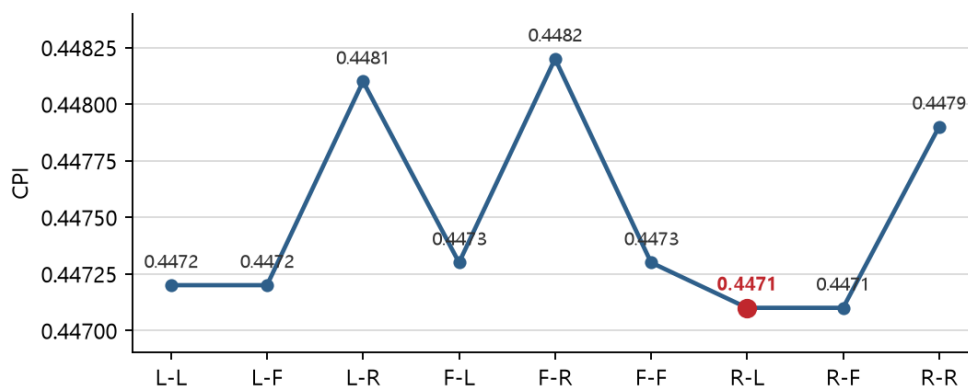
실행 결과 분석

세 가지 파라미터가 조합되었을 때 상호작용의 효과가 크며, 개별적인 최적 값보다 더 우수한 성능을 낸다는 것을 의미한다. 최적조합에서 가장 낮은 CPI가 기록된 이유는 Block Size, Associativity, Replacement Policy가 GCC 워크로드에 최적으로 맞물렸기 때문이다. GCC 워크로드에서 Block Size와 Associativity는 데이터를 효율적으로 배치하고 캐시 충돌을 줄이는 데 기여했지만, 교체 정책이 이를 최적으로 활용할 수 있도록 뒷받침하지 못하면 높은 CPI를 기록한다. 이러한 결과는 GCC와 같은 반복적 워크로드에서 Replacement policy의 조합이 성능 최적화에 핵심적인 역할을 한다는 점을 강조한다.

II. ANAGRAM

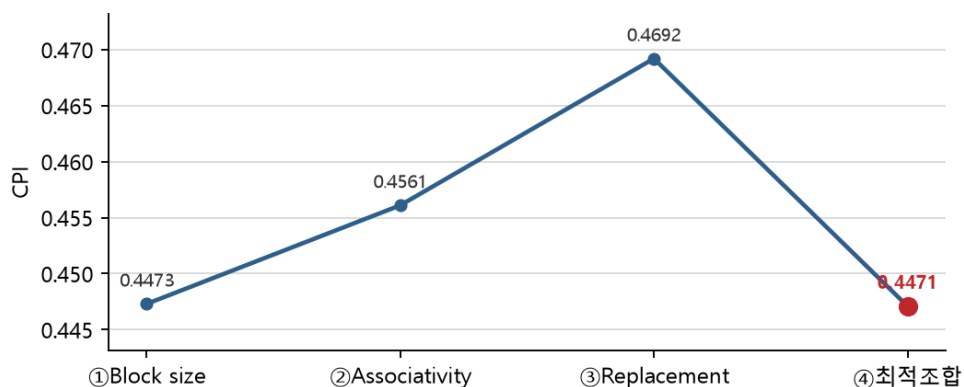
실험 번호	L1 교체 정책	L2 교체 정책	조합
1	LRU (l)	LRU (l)	L-L
2	LRU (l)	FIFO (f)	L-F
3	LRU (l)	RANDOM (r)	L-R
4	FIFO (f)	LRU (l)	F-L
5	FIFO (f)	RANDOM (r)	F-R
6	FIFO (f)	FIFO (f)	F-F
7	RANDOM (r)	LRU (l)	R-L
8	RANDOM (r)	FIFO (f)	R-F
9	RANDOM (r)	RANDOM (r)	R-R

ANAGRAM CPI (128B, 8-way, 9개 교체정책 조합)



Cache block size 128B, Associativity 8-way 고정 시 ANAGRAM CPI — R-L-R-F 조합이 0.4471로 동일하게 최소

ANAGRAM 개별 파라미터 vs 최적조합 CPI



①block size=128B	0.4473
------------------	--------

②associativity=8-way	0.4561
③replacement(L-R)	0.4692
④최적화 조건(3가지 결합)	0.4471

세 가지를 최적 조합으로 실행했을 때의 CPI가 가장 낮기 때문에 최적화에 성공했음을 확인할 수 있다.

실행 결과 분석

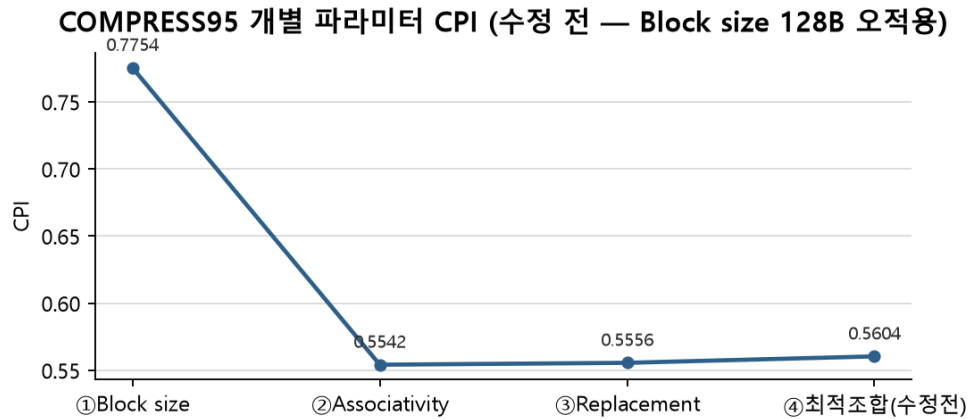
각 파라미터가 서로 보완적으로 작동하며 데이터 접근 패턴에 최적으로 대응했기 때문이다. Block size를 128B로 하면 데이터의 공간적 지역성을 효과적으로 활용해 데이터를 한 번에 읽어 들일 수 있지만, 단독으로는 캐시 충돌 문제를 해결하기 어렵다. 이 문제를 Associativity 8-way로 보완할 수 있으며, 데이터를 세트 내에서 유연하게 배치해 캐시 충돌을 줄일 수 있어 Block size의 효율성을 높이는 데 도움을 준다.

그러나 두 개의 파라미터만으로는 데이터를 유지·교체하는 관리가 부족할 수 있는데, 이 역할을 Replacement Policy로 보완할 수 있다. L1에서 Random 정책은 데이터를 무작위로 교체해 반복적 접근 패턴에서 충돌을 줄이며 데이터 다양성을 유지할 수 있고, L2에서 Random/FIFO 정책은 간단한 교체 방식으로 관리 오버헤드를 줄여 성능을 향상시킨다. 결론적으로 Anagram의 반복적이고 예측 가능한 데이터 접근 패턴에서 세 파라미터가 서로 보완적으로 작동하며 각자의 단점을 상쇄해 최적 조합에서 CPI가 가장 낮게 나타난다.

III. COMPRESS95

(1) trouble shooting

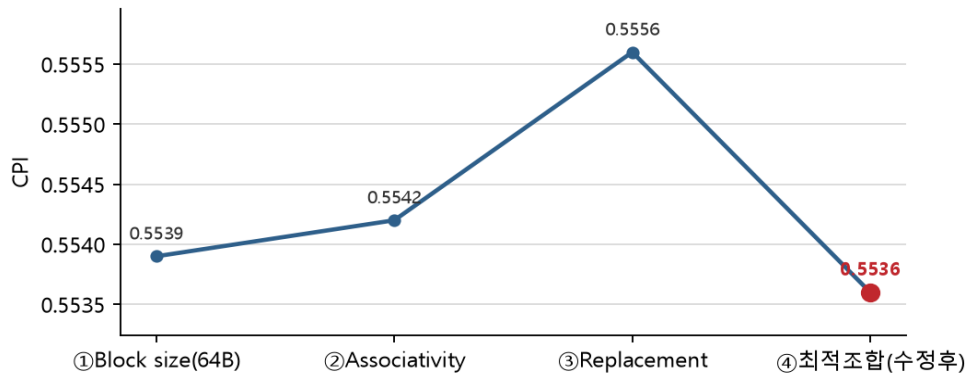
4가지 application 모두 최적화 조건이 같을 것으로 가정해 block size=128B로 simulation했다. 하지만 최적이라고 생각한 조건에서 오히려 성능 저하가 발생했다.



①block size=128B	0.7754
②associativity=8-way	0.5542
③replacement policy	0.5556
④최적화 조건 예상	0.5604

이는 COMPRESS95의 특성을 고려하지 않아 발생한 것으로 보인다. COMPRESS95는 64B일 때 가장 낮은 CPI를 보였고, 이것이 공간 지역성에 적합한 크기였다. 너무 작은 Block size(8B, 32B)에서는 캐시 miss가 자주 발생하고, 큰 Block size(128B 이상)는 불필요한 데이터를 가져오는 캐시 pollution을 증가시켜 성능을 저하시키는 것으로 분석했다. 따라서 64B Block size는 공간 지역성과 캐시 용량의 균형을 맞춘 최적의 크기로 작용하여 simulation했고 최적화에 성공했다.

COMPRESS95 개별 파라미터 vs 최적조합 CPI (수정 후)

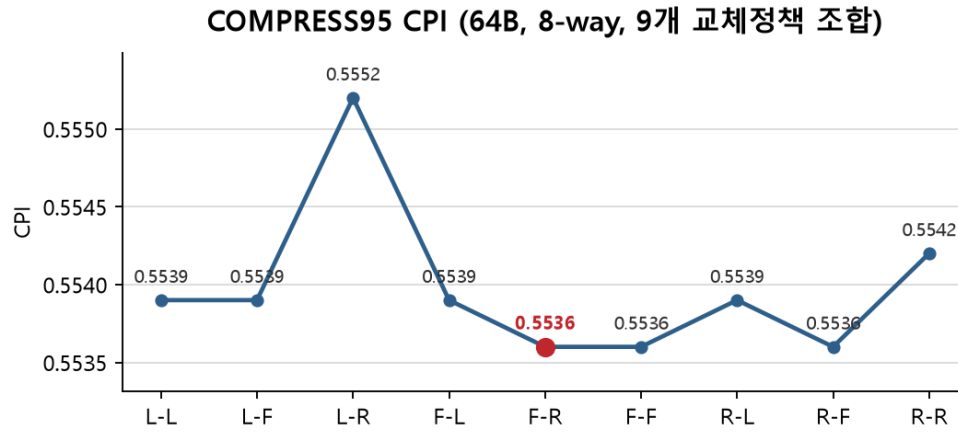


①block size=64B	0.5539
②associativity=8-way	0.5542
③replacement(L-L)	0.5556
④최적화 조건(수정 후)	0.5536

세 가지를 최적 조합으로 실행했을 때의 CPI가 가장 낮기 때문에 최적화에 성공했음을 확인할 수 있다.

(2) 실행 결과 그래프

실험 번호	L1 교체 정책	L2 교체 정책	조합
1	LRU (l)	LRU (l)	L-L
2	LRU (l)	FIFO (f)	L-F
3	LRU (l)	RANDOM (r)	L-R
4	FIFO (f)	LRU (l)	F-L
5	FIFO (f)	RANDOM (r)	F-R
6	FIFO (f)	FIFO (f)	F-F
7	RANDOM (r)	LRU (l)	R-L
8	RANDOM (r)	FIFO (f)	R-F
9	RANDOM (r)	RANDOM (r)	R-R



Cache block size 64B, Associativity 8-way 고정 시 COMPRESS95 CPI — R-L-R-F-L-F 조합이 0.5536으로 동일하게 최소

(3) 실행 결과 분석

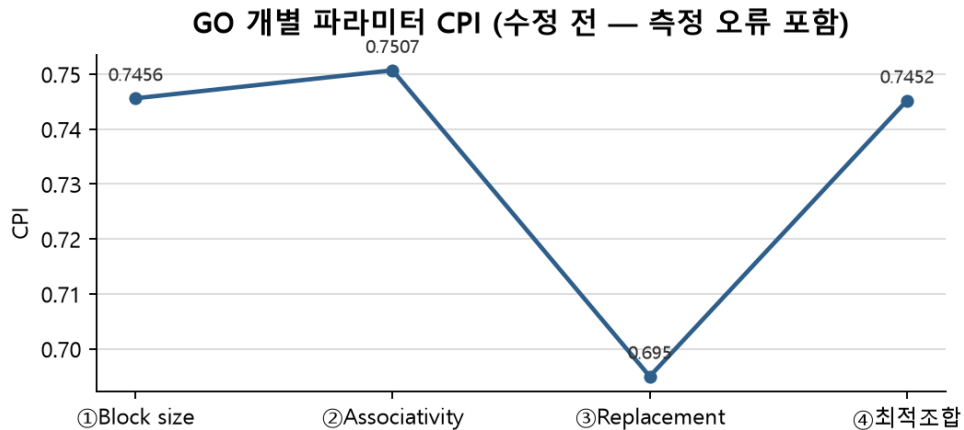
64B Block size + 8-way Associativity는 COMPRESS95의 데이터 접근 패턴에 최적화되어 낮은 miss rate 및 효율적인 데이터 유지를 가능하게 해주었으며, 교체 정책은 데이터 접근의 불규칙성을 완화하고 캐시 효율성을 유지하는 데 도움을 주었다. 특히 R-L, R-F, L-F 같은 정책은 LRU-FIFO 단독 정책의 단점을 보완하고 다양한 패턴에 잘 대응할 수 있는 장점을 보였다.

COMPRESS95는 데이터 접근 패턴이 복잡하고 캐시 라인을 고르게 사용하는 특성이 있다. 낮은 associativity에서는 동일한 cache 세트로 자주 mapping돼 conflict miss가 발생할 가능성이 크지만, 8-way는 적절히 높은 associativity로 데이터가 충분히 cache에 유지될 수 있게 해 미스율을 낮추고 성능을 개선한다. 다만 병렬 비교가 필요해 하드웨어 설계가 복잡해지고 비용·전력 소모가 증가하며, R-L-L-F 같은 혼합형 교체 정책은 사용 빈도·교체 우선순위 등의 관리 비용이 추가로 발생한다.

IV. GO

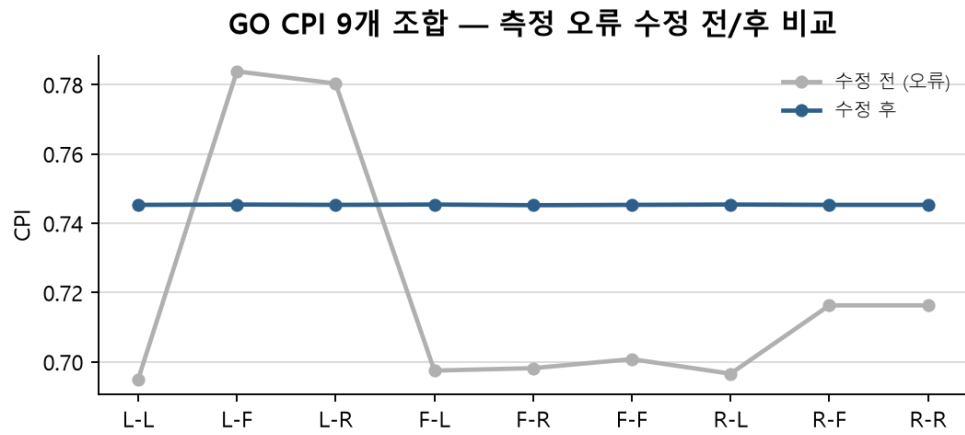
(1) trouble shooting

GO에서 위와 같은 방법으로 simulation했을 때, 최적의 조합이라고 예상했던 조합의 CPI가 가장 낮게 나오지 않았다. 이를 해결하기 위해 4번에서 진행한 simulation 결과의 정확성을 검토하기로 결정해, 팀원들 간의 교차 검증을 수행했다. 검증 과정에서 초기 simulation에서의 CPI 값에 오류가 있음을 발견했다.



①block size=128B	0.7456
②associativity=8-way	0.7507
③replacement(L-L, 측정 오류)	0.6950
④최적화 조건 예상	0.7452

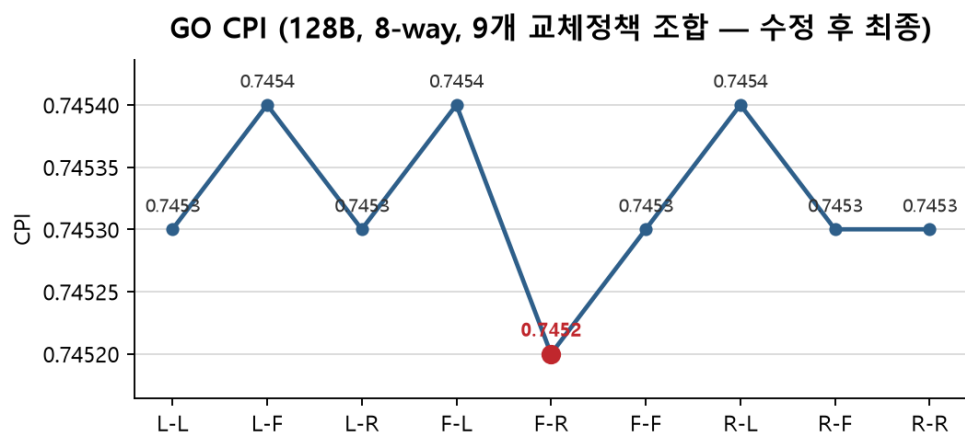
이는 첫 실행에서 config 파일을 수정하고 저장하는 과정에서 발생한 문제로 확인됐다. 수정된 CPI 값을 바탕으로 simulation을 재실행한 결과, 예상했던 최적의 조합에서 가장 낮은 CPI가 나타남을 확인할 수 있었다.



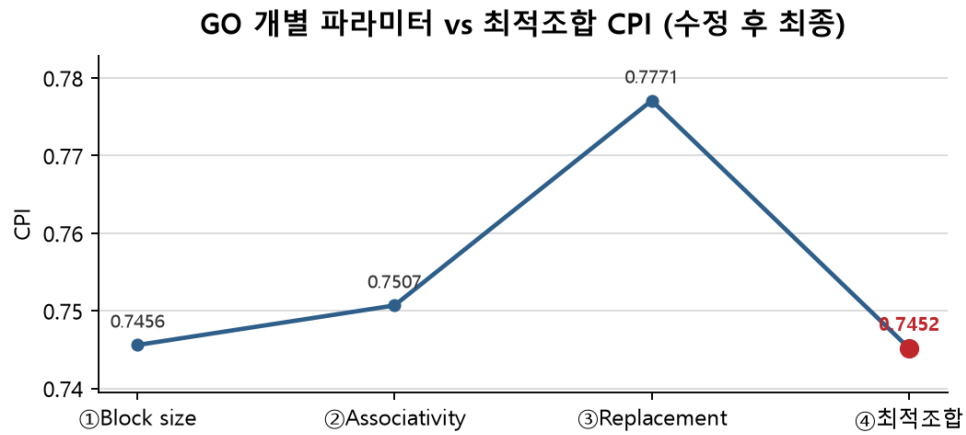
측정 오류 수정 전(회색)·후(파랑) 비교 — 수정 전에는 L-F/L-R 조합이 비정상적으로 높게 측정되었음

(2) 실행 결과 그래프 (수정 후)

실험 번호	L1 교체 정책	L2 교체 정책	조합
1	LRU (l)	LRU (l)	L-L
2	LRU (l)	FIFO (f)	L-F
3	LRU (l)	RANDOM (r)	L-R
4	FIFO (f)	LRU (l)	F-L
5	FIFO (f)	RANDOM (r)	F-R
6	FIFO (f)	FIFO (f)	F-F
7	RANDOM (r)	LRU (l)	R-L
8	RANDOM (r)	FIFO (f)	R-F
9	RANDOM (r)	RANDOM (r)	R-R



Cache block size 128B, Associativity 8-way 고정 시 GO CPI — F-R 조합이 0.7452로 최소



①block size=128B	0.7456
②associativity=8-way	0.7507
③replacement(L-L)	0.7771
④최적화 조건(수정 후 최종)	0.7452

세 가지를 최적 조합으로 실행했을 때의 CPI가 가장 낮기 때문에 최적화에 성공했음을 확인할 수 있다.

(3) 실행 결과 분석

본 조합은 캐시 miss율을 최소화하고 캐시 성능을 극대화하면서 복잡성과 오버헤드를 관리할 수 있는, 성능을 보이는 조합으로 판단된다.

GO application은 데이터 패턴이 예측하기 어렵고 불규칙적이다. Random 교체는 특정 패턴에 의존하지 않고 무작위로 교체하므로 예측 불가능한 접근 패턴에서 성능 저하를 방지할 수 있다. 또한 LRU는 최근에 사용된 데이터를 유지하므로 시간 지역성을 활용할 수 있다. Random과 LRU의 결합은 각각의 단점을 보완해 GO application에서 나타나는 다양한 패턴에 쉽게 대응할 수 있다. 다만 정교한 데이터 패턴이 존재할 경우 LRU나 FIFO 단일 혹은 조합보다 성능이 낮을 수 있다.

또한 GO application은 바둑 AI의 수 계산과 같이 트리 탐색과 데이터 연산이 많아 연속된 데이터 접근이 많다. 한 번에 큰 block(128B)을 가져오면 캐시 미스가 줄고 데이터 접근이 효율적이며, 이는 공간 지역성·캐시 miss rate 감소·캐시 오버헤드 절감으로 이어진다. 다만 block size가 증가함에 따라 필요한 데이터와 불필요한 데이터가 함께 캐시에 로딩돼 캐시 pollution이 발생하고, 중요한 데이터가 evict되어 메모리 대역폭 소모·miss penalty 증가로 이어질 수 있다.

8-way associativity는 conflict miss를 감소시키고 낮은 associativity에 비해 다양한 데이터가 캐시에 수

용될 수 있게 하지만, 비교 연산이 증가해 접근 시간·하드웨어 복잡성·전력 소모가 늘어나는 단점이 존재한다.